



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Vibe Coding with PyTorch Deep Learning

Course Code: C539

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	20 August 2026	Initial release. 4 topics, 21 hands-on labs, 2 days / 15 instructional hours.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Preparation.....	4
Topic 01 — AI Vibe Coding for PyTorch Fundamentals (~29% of course time).....	5
Lab 1 — What Is AI Vibe Coding — Your First PyTorch from a Prompt.....	6
Lab 2 — Setting Up Cursor, GitHub Copilot and Claude for PyTorch.....	8
Lab 3 — Prompting Patterns for Correct Deep Learning Code.....	10
Lab 4 — Vibe Coding PyTorch Tensor Operations.....	12
Lab 5 — Computation Graphs and Autograd with AI Assistance.....	15
Lab 6 — Reviewing and Debugging AI-Generated PyTorch Code.....	17
Topic 02 — Vibe Coding Neural Networks (~24% of course time).....	19
Lab 7 — Neural Network Architectures, Activation and Loss Functions.....	20
Lab 8 — Vibe Coding a Regression Model in PyTorch.....	22
Lab 9 — Vibe Coding a Classification Model with Softmax and Cross Entropy.....	25
Lab 10 — Generating Training Loops, Optimizers and Metrics from Prompts.....	27
Lab 11 — Saving, Loading and Iterating on Models.....	30
Topic 03 — Vibe Coding Convolutional Neural Networks (~24% of course time).....	32
Lab 12 — Overview of CNNs: Convolution, Pooling and Padding.....	33
Lab 13 — Vibe Coding a CNN Image Classifier.....	36
Lab 14 — Diagnosing Overfitting with AI Assistance.....	38
Lab 15 — Data Augmentation and Regularization via Prompts.....	41
Lab 16 — Transfer Learning with Pre-Trained Models.....	44
Topic 04 — Vibe Coding Recurrent Networks for Sequence Data (~23% of course time).....	47
Lab 17 — Overview of RNNs, LSTM and GRU.....	47
Lab 18 — Vibe Coding an LSTM for Time Series Forecasting.....	50
Lab 19 — Tuning Sequence Models with Follow-Up Prompts.....	53
Lab 20 — Evaluating and Visualizing Model Performance.....	56
Lab 21 — Packaging a Complete Deep Learning Project.....	59
Wrap-Up — What You Can Now Do.....	62
Next Steps.....	63
Glossary.....	64

Introduction

This Learner Guide accompanies the 2-day course AI Vibe Coding with PyTorch Deep Learning (C539), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 21 hands-on labs, organised into the 4 topics that follow the course slides and Lesson Plan. Across those 21 labs you build one project end to end — ForgeSight, a deep learning suite for a precision metal-parts factory — from an empty folder to a packaged, documented project, with an AI coding assistant writing the PyTorch alongside you.

Work through the labs in order: each one reuses the project folder, the data and the prompting habits established by the labs before it, and each lab states exactly which files it expects to already exist so you can rejoin if you fall behind. Every lab gives you a starting PROMPT to paste into your AI assistant (Cursor, GitHub Copilot Chat or Claude) and a 'Test it' step that tells you exactly what a correct result looks like. Read the generated code before you run it — deep learning code fails silently, and the point of this course is that you stay in control of the architecture, the loss and the evaluation rather than trusting the assistant to be right.

Course Learning Outcomes

- LO1: Explain what AI vibe coding is, set up Cursor, GitHub Copilot and Claude as PyTorch coding partners, and apply prompting patterns that produce correct deep learning code.
- LO2: Vibe code PyTorch tensor operations, computation graphs and autograd, and review and debug AI-generated PyTorch code rather than trusting it.
- LO3: Build regression and classification neural networks in PyTorch from prompts, choosing architectures, activation functions and loss functions deliberately.
- LO4: Generate training loops, optimizers and metrics from prompts, and save, load and iterate on trained models.
- LO5: Vibe code convolutional neural networks for image classification, diagnose overfitting, and apply data augmentation, regularization and transfer learning.
- LO6: Vibe code recurrent networks for sequence data, tune and evaluate them with follow-up prompts, and package a complete deep learning project.

Before You Start — Preparation

What you need

- A Windows or Mac laptop with at least 8 GB RAM and permission to install software. A GPU is NOT required — every lab is sized to run on CPU.
- Python 3.10 or newer — installed directly from python.org, or via Anaconda / Miniconda.
- PyCharm, Visual Studio Code, or Cursor if you prefer an AI-first editor.

- An AI coding assistant you can use in class: GitHub Copilot, Cursor's built-in assistant, or Claude in a browser tab.
- About 3 GB of free disk space — PyTorch, torchvision and the pre-trained ResNet weights are the largest downloads.
- A Google account for the Google Colab fallback, in case a local install fails.
- The course data from labs/resources/ (machines.csv and vibration_series.csv), plus the defect images you generate in Lab 12.

Verify your setup

Confirm your environment before Lab 1. This should print a version number for each library and finish with the line 'environment ready'. If any import fails, fix it before you continue — every later lab depends on this.

```
python -c "import sys, torch, torchvision, pandas, numpy, matplotlib;
print('python', sys.version.split()[0]); print('torch', torch.__version__);
print('torchvision', torchvision.__version__); print('environment ready')"
```

Conventions used in every lab

- Text shown as PROMPT is pasted into your AI coding assistant, not into a terminal.
- Text shown as COMMAND is typed into a terminal (PowerShell on Windows, Terminal on Mac).
- Placeholders such as <YOUR-NAME> are replaced with your own value.
- Every lab works inside a single course workspace folder, torch-vibe/, created in Lab 2.
- Random seeds are fixed at 42 throughout, so your numbers should be close to the guide — if they differ wildly, something is wrong.
- Every model runs on CPU by default; the device line is written so it uses a GPU automatically if you have one.
- Never paste a real API key, password or customer record into an AI assistant; the course data is synthetic for exactly this reason.

Topic 01 — AI Vibe Coding for PyTorch Fundamentals (~29% of course time)

What is AI vibe coding · Setting up Cursor, GitHub Copilot and Claude for PyTorch · Prompting patterns for correct deep learning code · Tensor operations · Computation graphs and autograd · Reviewing and debugging AI-generated PyTorch code

Key concepts

- Vibe coding means you describe the OUTCOME in plain language and the AI assistant writes the PyTorch. You keep the architecture, the loss, the shapes and the verification.

- Cursor, GitHub Copilot and Claude are AI pair programmers: they complete code inline, explain an unfamiliar torch argument, and refactor a working script on request.
- Deep learning code fails silently. A wrong loss still decreases, a wrong axis still returns a number, and a detached graph still trains to nothing, so 'it ran' is never the test.
- A good PyTorch prompt names five things: the tensor shapes, the task, the layers, the constraints and the expected output. Vague prompts return code that runs but learns nothing.
- A tensor is an n-dimensional array with a dtype, a device and optionally a gradient history. Almost every PyTorch bug is a shape, a dtype or a device bug.
- Autograd records every operation on a requires_grad tensor into a computation graph, then replays it backwards. `loss.backward()` fills `.grad`; the optimizer consumes it.
- Read the generated code before you run it and predict the printed shapes. Predicting wrongly is the fastest way to find the bug the assistant just wrote for you.

Lab 1 — What Is AI Vibe Coding — Your First PyTorch from a Prompt

Learning outcome: explain what AI vibe coding is and generate, read, run and verify your first PyTorch script from a plain-language prompt.

Goal: Meet the vibe coding loop before you install anything. You open Google Colab, where PyTorch is already available, and write a PROMPT instead of a line of code. The assistant returns a script that builds a tensor of ForgeSight machine readings, prints its shape and dtype, and computes a column mean. You read that code and predict every number it will print BEFORE you run it. Then you repeat the exercise with a deliberately vague prompt and compare what comes back. The gap between the two answers is the whole lesson: the assistant is only as precise as your framing.

What you'll build

A working `hello_torch.py` generated entirely from prompts, plus the first two entries in your `prompts.md` pattern library (Tools: Google Colab, PyTorch, Claude / ChatGPT / Copilot Chat.)

Step-by-step

1. Open Google Colab in your browser and start a new notebook. Nothing is installed today — Colab already has PyTorch, so the environment cannot be the thing that goes wrong. Go to <https://colab.research.google.com> and choose File > New notebook. You do not need a paid plan and you do not need a GPU — every lab in this course is sized to run on CPU. If Colab is blocked on your corporate network, tell the trainer now and pair with someone who can reach it; Lab 2 moves everything local.
2. Confirm PyTorch is present and note the version. Run this in the first Colab cell. You should see a version like 2.x.x. `torch.cuda.is_available()` printing False is

completely normal and expected — it simply means you are on CPU, which is what this course assumes throughout. PROMPT — paste this into your AI coding assistant:

```
import torch; print(torch.__version__); print(torch.cuda.is_available())
```

3. Write the VAGUE prompt first, in your AI assistant, and paste the result into a Colab cell. Do not fix anything yet — run it exactly as returned. Expect something generic and probably useless: a random tensor, no shapes printed, maybe numpy instead of torch. That is the point. Keep the output on screen so you can compare it in a moment. Resist the urge to improve the prompt — you will do that next, deliberately. PROMPT — paste this into your AI coding assistant:

Write some PyTorch code that works with machine sensor data.

4. Now write the SPECIFIC prompt. Notice it names the shape, the dtype, the operation and the exact expected output. Count what this prompt pins down: the shape (6, 3), the dtype (float32), the meaning of each column, five numbered outputs, the seed, and one explicit prohibition. That is the five-part pattern you will formalise in Lab 3 — shapes, task, layers/ops, constraints, expected output. PROMPT — paste this into your AI coding assistant: ```text Write a short PyTorch script called hello_torch.py for a factory dataset. It should:

5. Create a tensor of shape (6, 3) of float32 values representing 6 machine readings of (spindle_speed, coolant_temp, vibration_rms)

6. Print the tensor's shape, dtype and device on separate labelled lines

7. Print the mean of each of the 3 columns, labelled with the column name

8. Print the single reading with the highest vibration_rms

9. Use a fixed manual seed of 42 so the numbers are reproducible Do not use pandas or numpy - torch only. ```

10. Read the generated code line by line BEFORE running it. Write down, on paper, the shape it will print and how many numbers the column-mean line will produce. This is the habit the whole course rests on. Ask yourself: what does .mean(dim=0) return here, a scalar or three numbers? If you are not sure, that uncertainty is exactly what running the code is about to resolve — but predict first, then check. Being wrong here is useful; being wrong silently in Lab 18 is not.

11. Run the specific version in Colab and compare the output against your written prediction. If the printed shape is torch.Size([6, 3]) and you see three column means, the assistant read your prompt correctly. If it printed one number for the mean it used .mean() instead of .mean(dim=0) — a one-word correction, and a good first taste of refining rather than rewriting.

12. Start your prompt pattern library. Create prompts.md and paste BOTH prompts into it with a one-line note on what the specific one named that the vague one did not. A prompts.md that grows all course is worth more than any single script you

write today. You will add the formal pattern in Lab 3 and keep appending the prompts that worked through to Lab 21.

Test it

Your specific prompt produced a script that prints `torch.Size([6, 3])`, a float32 dtype, cpu as the device, THREE column means (not one), and one highest-vibration reading. Your written prediction of the shape matches what actually printed, and `prompts.md` contains both prompts with a note on the difference.

Note: Full commands, prompts and sample code are in `labs/lab01-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 2 — Setting Up Cursor, GitHub Copilot and Claude for PyTorch

Learning outcome: set up a local Python environment with PyTorch and an AI coding assistant that can see your project files.

Goal: Build the workspace every remaining lab runs in. You create the `torch-vibe/` folder and an isolated virtual environment, install PyTorch, torchvision, pandas and matplotlib, then open the folder in PyCharm, VS Code or Cursor and confirm your AI assistant is actually active and able to read your files. You finish by having the assistant generate `check_env.py` — a script that prints every version and ends with a single 'environment ready' line, so a broken install is obvious now rather than at 4pm tomorrow in the middle of the transfer-learning lab.

What you'll build

A `torch-vibe/` workspace with a virtual environment, PyTorch installed and verified, and an AI assistant that can see the project (Tools: Python 3.10+, PyTorch, torchvision, PyCharm / VS Code / Cursor, GitHub Copilot / Claude.)

Step-by-step

1. Create the course workspace and an isolated virtual environment so today's installs cannot break your other Python projects. On a Mac use `source .venv/bin/activate` instead of the Windows activate script. If you prefer Anaconda, `conda create -n torch-vibe python=3.11` then `conda activate torch-vibe` works just as well. The point is isolation — do not install into your system Python. Your prompt should change to `(.venv)` once activated. **COMMAND** — run this in your terminal:

```
mkdir torch-vibe
```

```
cd torch-vibe
```

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

2. Install the CPU build of PyTorch plus the libraries every lab uses. This is the largest download of the course. The `--index-url` flag selects the CPU-only wheels, which are a fraction of the size of the CUDA build and are all this course needs. On a slow connection this still takes several minutes — start it and read the next step while it downloads. If pip is very slow, add `--no-cache-dir`. **COMMAND** — run this in your terminal:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

```
pip install pandas matplotlib scikit-learn
```

3. Create the folder structure the whole ForgeSight project will use, and copy in the course data files. `data/` holds the ForgeSight CSVs and the images you generate in Lab 12, `models/` holds saved checkpoints from Lab 11 onward, and `reports/` holds the plots you produce in Lab 20. Copy `machines.csv` and `vibration_series.csv` from the course `labs/resources/` folder into `torch-vibe/data/` now. **COMMAND** — run this in your terminal:

```
mkdir data models reports
```

```
cd ..
```

4. Open the workspace in your editor and confirm your AI assistant is active — you should see inline suggestions or a chat panel that can reference your open files. In VS Code the Copilot icon in the status bar should not have a slash through it. In Cursor press `Ctrl+L` for chat. In PyCharm check the AI Assistant tool window. If none is available, keep Claude open in a browser tab and paste prompts there — every prompt in this course works in all four. **COMMAND** — run this in your terminal:

```
code torch-vibe
```

5. Prompt the assistant to write the environment check. Notice you are describing the **OUTPUT** you want, not the imports. Point 4 is the one learners skip. A bare `import torch` that fails gives a traceback; a named except gives `'torchvision is not installed'`, which is what you actually need at 9am with twenty people in the room. Point 3 matters because it proves the device string works, rather than just printing it. **PROMPT** — paste this into your AI coding assistant: ``text Create a file called `check_env.py` in this project. It should:

6. Print the Python version, then the installed versions of `torch`, `torchvision`, `pandas`, `matplotlib` and `sklearn`, one per labelled line

7. Print whether a CUDA GPU is available, and print which device the course will use (`'cuda'` if available else `'cpu'`)

8. Create a random tensor of shape (2, 3), move it to that device, and print its shape and device to prove the device line works

9. Wrap each import in a `try/except` that names the missing package clearly if it fails

10. Print exactly `'environment ready'` as the last line if everything succeeded ``

11. Read the generated code, then run it. Every version should print and the last line must read 'environment ready'. If an import fails, install just that package and re-run rather than reinstalling everything. If torch itself fails to import on Windows, it is almost always a 32-bit Python — check that `python -c "import platform; print(platform.architecture())"` says 64bit. **COMMAND** — run this in your terminal:

```
python check_env.py
```

12. Freeze the exact versions you installed, so the project is reproducible on another machine later in Lab 21. `requirements.txt` now pins the exact versions that work on your machine. Lab 21 turns this into part of the packaged project; a saved model is only reloadable against the library versions it was written with. **COMMAND** — run this in your terminal:

```
pip freeze > requirements.txt
```

Test it

`'python check_env.py'` prints a version number for Python, torch, torchvision, pandas, matplotlib and sklearn, prints the device as 'cpu' (or 'cuda' if you have a GPU), prints a tensor of shape `torch.Size([2, 3])` on that device, and ends with exactly the line 'environment ready'. `requirements.txt` exists and contains a pinned torch version.

Note: Full commands, prompts and sample code are in `labs/lab02-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 3 — Prompting Patterns for Correct Deep Learning Code

Learning outcome: apply a repeatable five-part prompting pattern that produces correct, runnable deep learning code.

Goal: Turn yesterday's lucky prompt into a method. You formalise the five-part pattern — shapes, task, layers and operations, constraints, expected output — and then test it under pressure on a genuinely shape-sensitive task: loading the ForgeSight telemetry and reshaping it into batches. You run a controlled A/B: the same task prompted vaguely and prompted with the pattern, and you diff the two results. You then practise the follow-up prompt, which is the skill that actually saves time — feeding back a specific symptom instead of the word 'fix'.

What you'll build

A `prompts.md` pattern library with the five-part template and a worked A/B comparison, plus a working `load_data.py` (Tools: PyTorch, pandas, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Write the five-part pattern into prompts.md as a reusable template you will fill in for every later lab. The template to write down: SHAPES (what data exists, what shape, what dtype) · TASK (what the code must achieve) · OPERATIONS (the specific layers, transforms or steps) · CONSTRAINTS (what it must NOT do, seeds, libraries allowed) · EXPECTED OUTPUT (the function signature and what it prints). You will fill in all five for every substantial prompt from here on.
2. Prompt the task VAGUELY first, and keep the result in a scratch file. You are building evidence, not code. Expect the vague version to invent column names, guess at the target, split randomly with no seed, and very likely standardise before splitting. Every one of those is a real defect and none of them raises an error. Save it as scratch_vague.py — you are going to diff it in a moment. PROMPT — paste this into your AI coding assistant:

Load the machine data and get it ready for a PyTorch model.

3. Now prompt the SAME task using the five-part pattern. Every clause maps to one part of the template. This is a long prompt and that is the point. It takes about ninety seconds to write and it replaces four rounds of correction. Notice CONSTRAINTS is where the real expertise sits: 'do not fit on the full dataset' is the sentence that prevents the leakage bug this entire lab is built around. PROMPT — paste this into your AI coding assistant:

Create load_data.py for a PyTorch project.

SHAPES: data/machines.csv has 4000 rows and these columns - machine_id, spindle_speed, feed_rate, coolant_temp, vibration_rms, spindle_load, ambient_temp, tool_wear_um, qc_class.

TASK: load the CSV and return PyTorch tensors ready for supervised learning.

OPERATIONS: drop machine_id; use the 6 sensor columns as features X; return tool_wear_um as a float32 regression target and qc_class as an int64 classification target; split 70/15/15 into train/val/test with a fixed seed of 42.

CONSTRAINTS: standardise the features using the TRAINING split's mean and std only, then apply those same statistics to val and test - do not fit on the full dataset. Return the mean and std so they can be reused later.

EXPECTED OUTPUT: a function load_forgesight(path) returning a dict with keys X_train, y_wear_train, y_qc_train and the same for val and test, plus feat_mean and feat_std. When run as a script it prints the shape and dtype of every tensor.

4. Read the generated code and check ONE thing specifically: where is the mean and standard deviation computed? Find that line before you run anything. Look for X_train.mean(0) or a StandardScaler fitted after the split — correct. If you find the mean computed on the full X before the split, the assistant leaked, even though you told it not to. Assistants get this wrong often enough that checking it is a permanent habit, not a one-off.

5. Run it and confirm every shape and dtype is what the prompt asked for. Expected: X_train around torch.Size([2800, 6]) float32, X_val and X_test around torch.Size([600, 6]), y_wear_ float32 of matching length, y_qc_ int64. If y_qc is float32 the assistant missed the dtype — CrossEntropyLoss in Lab 9 will reject it, so fix it now with a follow-up. COMMAND — run this in your terminal:

```
python load_data.py
```

6. Practise the follow-up prompt. Feed back a precise symptom rather than asking it to 'fix it', using this template with whatever your script actually got wrong. Compare this to typing 'fix it'. You named the file, the symptom, the cause, the required change and the scope ('change only the standardisation block'). That last clause is what stops the assistant helpfully rewriting your whole file and losing the parts that already worked. PROMPT — paste this into your AI coding assistant:

```
In load_data.py the features are standardised using the mean and std of the whole dataset before the split, which leaks validation and test information into training. Recompute mean and std from X_train only, then apply those same values to val and test. Change only the standardisation block.
```

7. Append both prompts and the corrected result to prompts.md, with a note naming the specific defect the pattern caught. Your prompts.md is now genuinely useful — the template plus one worked example of the pattern catching a silent bug. Keep appending to it. By Lab 21 it is the most portable thing you take home.

Test it

`python load\_data.py` prints X_train torch.Size([2800, 6]) float32, y_qc_train int64, and matching val/test shapes. You can point at the exact line where mean and std are computed and confirm it uses X_train only. prompts.md holds the five-part template, both A/B prompts and the follow-up prompt.

Note: Full commands, prompts and sample code are in labs/lab03-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 4 — Vibe Coding PyTorch Tensor Operations

Learning outcome: create, reshape, broadcast, index and move tensors, and read a shape error well enough to know which axis is wrong.

Goal: Tensors are where almost every PyTorch bug actually lives, so you meet them deliberately. You prompt the assistant for a tensor workout script that runs over the real ForgeSight telemetry: dtype and device, reshape versus view, broadcasting, reduction along a chosen axis, boolean masking and matrix multiplication. Each section prints a shape, and each shape you predict first. You then trigger two shape errors on purpose

and practise reading the message — because the error text tells you exactly which axis disagreed, if you know how to read it.

What you'll build

A `tensor_ops.py` workout over the real telemetry, plus a `shape_errors.py` that demonstrates and explains two classic shape failures (Tools: PyTorch, pandas, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Prompt for the tensor workout. Note that every section is required to PRINT a shape — you cannot verify what you cannot see. Section 3 is subtle and worth reading twice: `.view()` requires contiguous memory and fails after a transpose, `.reshape()` silently copies instead. If the assistant does not actually produce a non-contiguous tensor (usually by transposing first), the demo is fake — follow up and say so. PROMPT — paste this into your AI coding assistant:

Create `tensor_ops.py` using the `load_forgesight` function from `load_data.py`.

SHAPES: `X_train` is a float32 tensor of shape (2800, 6); `y_wear_train` is float32 of shape (2800,).

TASK: demonstrate the core tensor operations on this real data, printing the shape and dtype after every step.

OPERATIONS: (1) print shape, dtype, device and number of elements of `X_train`; (2) reshape `y_wear_train` to a column vector (2800, 1) and explain in a comment why a regression target usually needs this; (3) show the difference between `.view()` and `.reshape()` on a non-contiguous tensor; (4) compute the per-feature mean with `dim=0` and the per-sample mean with `dim=1` and print both shapes; (5) broadcast-subtract the per-feature mean from `X_train` and print the resulting shape; (6) use a boolean mask to select the rows where `vibration_rms` (column index 3) is above its mean, and print how many rows survived; (7) matrix-multiply `X_train` by a random weight tensor of shape (6, 1) and print the output shape.

CONSTRAINTS: torch only, manual seed 42, no training and no gradients.

EXPECTED OUTPUT: labelled print lines for every step above.

2. Before running, write down your predicted answer to three questions: what shape does `dim=0` mean give, what shape does `dim=1` give, and what shape comes out of the matmul? The answers: `dim=0` reduces ACROSS rows and gives one number per feature, so `torch.Size([6])`; `dim=1` reduces across features and gives one number per sample, `torch.Size([2800])`; the matmul gives `torch.Size([2800, 1])`. The rule worth memorising is that the dim you name is the one that disappears.
3. Run it and check your three predictions against the output. If a prediction was wrong, that is the most valuable line of output on your screen today. Note which one in `prompts.md` — `dim=0` versus `dim=1` is the single most common confusion in the

room, and it is the reason a per-feature normalisation sometimes silently normalises per-sample instead. **COMMAND** — run this in your terminal:

```
python tensor_ops.py
```

4. Now break it deliberately. Prompt for a script that causes two classic shape errors and explains each one. Error 2 is the whole point of this lab. A prediction of shape (32, 1) against a target of shape (32,) broadcasts to a (32, 32) matrix of pairwise differences, and MSELoss cheerfully averages all 1024 of them. It does not raise. Your training loop will run, your loss will decrease, and your model will be wrong. **PROMPT** — paste this into your AI coding assistant:

```
Create shape_errors.py that deliberately triggers two common PyTorch shape errors and catches each one.
```

```
ERROR 1: matrix-multiply a tensor of shape (2800, 6) by a tensor of shape (1, 6) so the inner dimensions do not match.
```

```
ERROR 2: compute MSELoss between a prediction of shape (32, 1) and a target of shape (32,), which does NOT raise but silently broadcasts to a (32, 32) result and returns a wrong loss.
```

```
For each: wrap it in try/except, print the full error message for error 1, and for error 2 print the shape of the broadcast result and the wrong loss value next to the correct loss when the target is reshaped to (32, 1).
```

```
Add a comment under each explaining which axis disagreed and the one-line fix.
```

5. Run it and read Error 1's message carefully. Identify in the text exactly which two numbers had to match and did not. The message names the mismatched dimensions explicitly — something like 'mat1 and mat2 shapes cannot be multiplied (2800x6 and 1x6)'. The inner numbers, 6 and 1, are the ones that must agree. Once you know to read the two shapes in that sentence, most shape errors take about five seconds to diagnose. **COMMAND** — run this in your terminal:

```
python shape_errors.py
```

6. Study Error 2 — it is the dangerous one, because nothing raises. Confirm the two loss numbers differ and note which one is correct. Expect the broadcast loss to be substantially larger and completely meaningless. The fix is `target.view(-1, 1)` — or `pred.squeeze()`. Add a line to `prompts.md`: 'always print `pred.shape` and `target.shape` before the first loss call'. You will thank yourself in Lab 8.

Test it

`tensor_ops.py` prints `torch.Size([6])` for the `dim=0` mean, `torch.Size([2800])` for the `dim=1` mean and `torch.Size([2800, 1])` for the `matmul`, and your written predictions match. `shape_errors.py` prints a caught error naming the (2800x6 and 1x6) mismatch, and shows a (32, 32) broadcast result whose loss value visibly differs from the correct (32, 1) loss.

Note: Full commands, prompts and sample code are in labs/lab04-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 5 — Computation Graphs and Autograd with AI Assistance

Learning outcome: explain what autograd records, verify a gradient by hand, and show what detach and no_grad actually change.

Goal: Autograd is the machinery that makes every later lab possible, so you verify it rather than trust it. You prompt for a script that builds a tiny expression on a `requires_grad` tensor, calls `backward()`, and prints the gradient next to the value you derive analytically on paper — they must match to several decimal places. You then investigate the three ways gradients silently stop flowing: `detach()`, a `no_grad()` block, and a tensor created without `requires_grad`. Finally you watch gradients ACCUMULATE across two backward calls, which is exactly the bug a missing `zero_grad()` causes in a training loop.

What you'll build

An `autograd_lab.py` proving a hand-derived gradient, plus a demonstration of `detach`, `no_grad` and gradient accumulation (Tools: PyTorch autograd, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Derive the gradient on paper FIRST. For $y = 3x^2 + 2x + 1$ at $x = 4$, write down dy/dx by hand before you write any prompt. $dy/dx = 6x + 2$, so at $x = 4$ the gradient is 26. Do this on paper before you run anything. The habit of having an expected number before you look at the output is the difference between verifying and hoping.
2. Prompt for the verification script. You are asking the assistant to prove your arithmetic, not to teach you calculus. Part 4 is the one that matters most for the rest of the course, so make sure the assistant actually implements it as described — two `backward()` calls on a freshly rebuilt expression, without zeroing. Some assistants 'helpfully' insert the `zero_grad` and destroy the demonstration. If yours does, follow up and insist the first version omits it. PROMPT — paste this into your AI coding assistant:

Create `autograd_lab.py` demonstrating PyTorch autograd.

```
PART 1 - verify a gradient: create x = torch.tensor(4.0, requires_grad=True),
compute y = 3*x**2 + 2*x + 1, call y.backward(), and print x.grad. Also print the
analytic value 6*x + 2 computed manually, and assert the two agree to 5 decimal
places.
```

PART 2 - inspect the graph: print `y.requires_grad`, `y.grad_fn` and `x.is_leaf`, with a comment explaining what each one tells you.

PART 3 - three ways gradients stop: (a) repeat Part 1 but call `.detach()` on `x` before the expression, (b) repeat it inside a `torch.no_grad()` block, (c) repeat it with `requires_grad` left as `False`. For each, print whether `grad_fn` exists and whether `x.grad` is `None`, and add a comment on when you would WANT each behaviour.

PART 4 - accumulation: with a fresh `x`, call `backward()` twice on the same expression without zeroing, printing `x.grad` after each call. Then show that `x.grad.zero_()` between calls gives the correct value both times.

CONSTRAINTS: torch only, no training loop, no `nn.Module`. Label every printed line.

3. Read the code, then run Part 1 and compare the printed gradient against your paper answer. `x.grad` must print 26.0 (or 26.000000...). If the assert fires, read which side is wrong — usually the assistant transcribed the analytic derivative incorrectly, which is itself a nice illustration that the assistant is not automatically right about mathematics. COMMAND — run this in your terminal:

```
python autograd_lab.py
```

4. Study Part 2's output. Note which tensor has a `grad_fn` and which is a leaf — this is the shape of the computation graph in miniature. `y.grad_fn` shows something like `<AddBackward0>` — that is the last operation recorded in the graph, and it is the thread PyTorch pulls to walk backwards. `x.is_leaf` is `True` because you created `x` directly; `y` is not a leaf because it was computed. Only leaf tensors with `requires_grad` get a populated `.grad`.
5. Work through Part 3 and answer for yourself: which of the three would you actually use at evaluation time, and which one is a bug when it appears in a model? `no_grad()` is what you want at evaluation and inference — it is correct and it saves memory. `detach()` is what you want when deliberately cutting a graph, for example to stop a gradient flowing into a target. `requires_grad=False` on something that should be learning is almost always a bug.
6. Study Part 4 closely. Note the exact value printed after the second `backward()` call and compare it to the first. Expect 26.0 then 52.0. Nothing raises, nothing warns — the gradient simply doubled. In a training loop this means every batch after the first applies an update built from the sum of all previous gradients, so the model moves too far in a stale direction. This is why `zero_grad()` comes first in the five-line loop.
7. Add the accumulation finding to `prompts.md` as a checklist item for reviewing any AI-generated training loop. Write it as a review question you will apply to generated code: 'Is `optimizer.zero_grad()` present, and is it before the backward call rather than after?' You will use this exact check in Lab 6 and Lab 10.

Test it

Part 1 prints `x.grad` as 26.0 and the assertion against your hand-derived $6x+2$ passes. Part 2 shows `y` has a `grad_fn` while `x` is a leaf. Part 3 shows `grad_fn` absent and `x.grad` None in all three cases. Part 4 prints 26.0 after the first backward and 52.0 after the second, then 26.0 both times once `zero_()` is called.

Note: Full commands, prompts and sample code are in `labs/lab05-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 6 — Reviewing and Debugging AI-Generated PyTorch Code

Learning outcome: review AI-generated PyTorch against a fixed checklist and correct each defect with a targeted follow-up prompt.

Goal: Everything so far has been about producing code. This lab is about refusing to trust it. You are given `train_wear_buggy.py` — a script that runs to completion, prints a decreasing loss and reports believable-looking numbers, while containing five real defects. Nothing crashes. You review it against a written checklist, predict what each defect does to the result, then fix them one at a time with targeted follow-up prompts, re-running after each fix so you can see which defect was costing what. This is the single most transferable skill in the course.

What you'll build

A corrected `train_wear.py`, a `review_notes.md` recording all five defects, and a reusable AI PyTorch review checklist (Tools: PyTorch, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Copy the buggy script from the course resources into your workspace and run it BEFORE reading it. Note the final loss and score it reports. Write the numbers down — the reported MAE in microns and the QC accuracy. The entire lesson lands only if you have the 'before' figures in front of you when you see the 'after'. Note also that the script never prints a baseline, which is itself the sixth defect nobody asked you to look for. **COMMAND** — run this in your terminal:

```
python train_wear_buggy.py
```

2. Now read it against the checklist without running anything. Write your suspicions into `review_notes.md` before you ask the assistant anything. The checklist, which becomes your permanent one: (1) Is any scaling or fitting done before the split? (2) Is `optimizer.zero_grad()` present and before `backward()`? (3) Does the output layer apply an activation that the loss also applies? (4) Is evaluation inside `model.eval()` and `torch.no_grad()`? (5) Do prediction and target shapes match exactly? Most learners find three of the five unaided — that is a good score.

3. Ask the assistant to review it — but constrain the review so it reports rather than rewrites. The 'do NOT rewrite' constraint matters. An unconstrained assistant returns a fixed file, you accept it, and you learn nothing about what was wrong. Forcing it to report line-by-line keeps you as the reviewer and makes the assistant explain itself. Ranking by distortion teaches you which bugs actually matter. **PROMPT** — paste this into your AI coding assistant:

Review the PyTorch script `train_wear_buggy.py` for correctness defects. Do NOT rewrite the file.

For each defect, report exactly three things: (1) the line, quoted verbatim; (2) what it does to the RESULT - be specific about whether it makes the score too high, too low, or meaningless; (3) the minimal one-line fix.

Check specifically for: standardisation or scaling fitted before the train/test split; a missing or misplaced `optimizer.zero_grad()`; a softmax or sigmoid applied before a loss function that already applies it; evaluation performed without `model.eval()` or `torch.no_grad()`; a target tensor whose shape does not match the prediction shape.

Rank the defects by how much they distort the reported result.

4. Compare the assistant's list against your own. Anything you found that it missed is worth more than anything it found that you missed — record both in `review_notes.md`. Assistants are good at the shape and `zero_grad` defects and noticeably weaker at leakage, because leakage is about the ORDER of operations rather than any single wrong line. That asymmetry is exactly why the human review step survives.

5. Fix the defects ONE AT A TIME, re-running after each. Start with the one the assistant ranked most distorting. One at a time, re-running each time, is the discipline. Fix all five at once and you have no idea which one was responsible for the inflated score — and in a real project that is the question you will be asked. **PROMPT** — paste this into your AI coding assistant:

In `train_wear_buggy.py`, the features are standardised using the mean and standard deviation of the full dataset before the train/test split, which leaks test information into training. Recompute the mean and std from the training split only and apply those same values to the test split. Change only that block and save the result as `train_wear.py`.

6. Continue with the remaining defects, using the same specific-symptom style. After each fix, record the new loss and score in `review_notes.md`. The five defects and their effects: leakage before the split inflates the score; the missing `zero_grad` makes updates too large and training unstable; the extra softmax before `CrossEntropyLoss` flattens the gradients so the model learns slowly or not at all; evaluating without `eval()/no_grad()` leaves dropout on and wastes memory, giving a noisy and pessimistic score; the (N,) versus (N,1) target mismatch silently broadcasts and produces an entirely wrong loss.

7. Compare the original reported numbers against the corrected ones, and write one sentence in `review_notes.md` explaining why the buggy script's numbers looked believable while both models were actually stuck at their baselines. The answer that matters: the numbers were believable, not impressive. A tool-wear MAE in the low tens of microns sounds like a working model until you compare it with simply predicting the average — which is all the broadcast loss actually trained it to do. The QC accuracy looks respectable for the same reason: it is roughly the share of 'ok' parts, so the model is predicting 'ok' for everything. Write that in your own words. A number is only meaningful next to its baseline, which is why every lab from here on computes the baseline FIRST.

Test it

`review_notes.md` lists all five defects with the line quoted, the effect on the result and the fix. `train_wear.py` runs clean, and its corrected tool-wear MAE is clearly BETTER than the buggy script's — which was no better than predicting the average — while the corrected QC accuracy rises above the share of 'ok' parts the buggy version was merely echoing. You can state in one sentence why the buggy numbers looked believable, and your reusable checklist is written down in `prompts.md`.

Note: Full commands, prompts and sample code are in `labs/lab06-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Topic 02 — Vibe Coding Neural Networks (~24% of course time)

Neural network architectures, activation and loss functions · Vibe coding a regression model · Vibe coding a classification model with softmax and cross entropy · Generating training loops, optimizers and metrics from prompts · Saving, loading and iterating on models

Key concepts

- A neural network is a stack of linear layers separated by non-linearities. Without the activation function the whole stack collapses into one linear layer.
- ReLU is the default hidden activation; sigmoid and tanh saturate and stall learning in deep stacks. The OUTPUT layer's activation is decided by the task, not by taste.
- The loss function must match the output layer: `MSELoss` for regression, `CrossEntropyLoss` for multi-class. Mismatch them and the model trains happily towards nonsense.
- `nn.CrossEntropyLoss` applies log-softmax internally, so the model must output RAW LOGITS. Adding a softmax layer before it is the single most common PyTorch bug.

- The training loop is always the same five lines: zero the gradients, forward, compute loss, backward, step. Forgetting `zero_grad()` accumulates gradients across batches.
- Optimizers differ in how they use the gradient: SGD follows it, Adam adapts a per-parameter learning rate. Learning rate matters more than the choice of optimizer.
- Save the `state_dict`, not the pickled object, and record the architecture alongside it - weights without the class that built them are unloadable.

Lab 7 — Neural Network Architectures, Activation and Loss Functions

Learning outcome: choose hidden and output activations and match the loss function to the output layer.

Goal: Before you train anything, settle the three decisions that determine whether training can work at all: the architecture, the activation and the loss. You prompt for a script that plots ReLU, sigmoid and tanh next to their gradients, then proves the claim that a stack of linear layers without activations collapses into a single linear layer — by showing two networks produce identical outputs. You finish by building a decision table linking each task type to its output layer and its loss, which becomes the reference you use for the next fourteen labs.

What you'll build

An `activations.py` with plotted activations and a linear-collapse proof, plus a `losses_cheatsheet.md` decision table (Tools: PyTorch, matplotlib, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Prompt for the activation comparison. Ask for the gradients as well as the functions — the gradient is what explains the behaviour. The saturated-fraction numbers are the quantitative version of what the plot shows. Sigmoid and tanh flatten at both ends, so their gradients approach zero over a large part of the range; ReLU has gradient exactly 1 for every positive input. That is why ReLU became the default hidden activation. PROMPT — paste this into your AI coding assistant:

Create `activations.py` for a PyTorch course.

TASK: compare the three common activation functions and show why the choice matters.

OPERATIONS: (1) create `x = torch.linspace(-6, 6, 200)`; (2) compute ReLU, sigmoid and tanh of `x` using `torch.nn.functional`; (3) compute the gradient of each with respect to `x` using `autograd`; (4) plot a 2-row figure - top row the three activations, bottom row their three gradients, sharing the `x` axis - and save it to `reports/activations.png`; (5) print, for each activation, the fraction of the input range where its gradient is smaller than 0.01.

CONSTRAINTS: torch and matplotlib only, no seaborn, label every axis and subplot.

EXPECTED OUTPUT: the saved figure plus three printed 'saturated fraction' lines.

2. Run it, open the figure, and answer one question from the bottom row: which activation keeps a usable gradient across the widest input range? The answer is ReLU. Sigmoid's gradient peaks at 0.25 and decays fast, so in a deep stack the gradients multiply down towards nothing — the vanishing gradient problem you will meet again with plain RNNs in Lab 17. Note that ReLU's flat negative half is a real cost (dead units), just a smaller one. **COMMAND** — run this in your terminal:

```
python activations.py
```

3. Now prove the linear-collapse claim. This is the reason activations exist at all. Composing the layers means multiplying the weight matrices in the right order and carrying the biases through: $W = W_3 @ W_2 @ W_1$ and $b = W_3 @ (W_2 @ b_1 + b_2) + b_3$, give or take the transpose convention PyTorch uses. If the assistant gets the order wrong the assertion fails — that is a real bug worth making it fix rather than accepting a loosened tolerance. **PROMPT** — paste this into your AI coding assistant:

```
Add a section to activations.py called linear_collapse().
```

TASK: prove that a stack of linear layers with no activation is equivalent to a single linear layer.

OPERATIONS: (1) build `net_a = nn.Sequential(nn.Linear(6, 32), nn.Linear(32, 16), nn.Linear(16, 1))` with manual seed 42; (2) analytically collapse its three weight matrices and biases into ONE equivalent `nn.Linear(6, 1)` by composing them; (3) run the same random input batch of shape (8, 6) through both and print the two outputs side by side plus their maximum absolute difference; (4) assert the difference is below $1e-4$; (5) repeat with `nn.ReLU()` inserted between the layers and print the maximum absolute difference now.

EXPECTED OUTPUT: near-zero difference without activations, clearly non-zero difference with ReLU.

4. Run it and confirm the assertion passes. Read the second difference — that number is what the non-linearity is buying you. A difference below $1e-4$ without activations means three layers did exactly as much as one: 561 parameters achieving what 7 could. With ReLU the difference is large, because the network can now bend. If the assistant 'fixes' a failing assert by raising the tolerance to 1.0, reject it and make it fix the algebra. **COMMAND** — run this in your terminal:

```
python activations.py
```

5. Build the decision table by hand in `losses_cheatsheet.md`. Fill it in yourself before checking it with the assistant. Fill in three rows: task type, final layer, output activation, loss function. Do it from memory. The row people get wrong is multi-class — write down what you believe before you check, so you find out whether you actually knew it.

6. Check your table against the assistant, and specifically interrogate the classification row. The critical answer: `nn.CrossEntropyLoss` applies log-softmax internally and `nn.BCEWithLogitsLoss` applies sigmoid internally. Adding your own softmax before `CrossEntropyLoss` applies it twice, which flattens the distribution,

shrinks the gradients and makes the model learn slowly or not at all — while still running and still printing a falling loss. This is the single most common AI-generated PyTorch bug. PROMPT — paste this into your AI coding assistant:

I have written a table mapping task type to output layer and PyTorch loss function:

- Regression (one continuous value) -> Linear(h, 1), no activation -> nn.MSELoss
- Binary classification -> Linear(h, 1), no activation -> nn.BCEWithLogitsLoss
- Multi-class classification (4 classes) -> Linear(h, 4), no activation -> nn.CrossEntropyLoss

For each row, confirm or correct it, and explain in one sentence what happens numerically if someone adds a softmax or sigmoid to the output layer before these losses. Be specific about which losses already apply that function internally.

7. Record the answer in losses_cheatsheet.md in your own words, especially the softmax warning. Write the warning in your own words, not copied. Something like: 'CrossEntropyLoss wants raw logits. If I see nn.Softmax in a model's output layer next to CrossEntropyLoss, one of them must go.' You will apply this exact check in Lab 9, where the assistant will very likely make the mistake for you.

Test it

reports/activations.png shows three activations over three gradients, and the printed saturated fractions are near zero for ReLU and clearly non-zero for sigmoid and tanh. linear_collapse() asserts a difference below 1e-4 without activations and prints a visibly larger difference with ReLU. losses_cheatsheet.md has all three rows and states which losses apply softmax or sigmoid internally.

Note: Full commands, prompts and sample code are in labs/lab07-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 8 — Vibe Coding a Regression Model in PyTorch

Learning outcome: build, train and honestly evaluate a regression network that predicts a continuous value.

Goal: Build ForgeSight's first real model: a network predicting tool wear in microns from six sensor readings. You prompt for an nn.Module with a deliberate architecture, an MSELoss, and a training loop that reports train and validation loss each epoch. Crucially you establish a BASELINE first — predicting the training mean for every row — because a regression score means nothing on its own. You then check the prediction and target shapes explicitly, which is where the (N,) versus (N,1) trap from Lab 4 comes back to bite anyone who skipped it.

What you'll build

A trained tool-wear regression network reporting MAE and RMSE that beat the mean baseline (Tools: PyTorch, nn.Module, MSELoss, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Compute the baseline FIRST, before any model exists. You cannot judge a regression score without it. A mean-predictor is the honest floor for regression. Its RMSE is essentially the target's standard deviation, because that is what standard deviation measures. If your network cannot beat this, it has learned nothing, no matter how impressive the loss curve looks. PROMPT — paste this into your AI coding assistant:

```
Create baseline_wear.py. Using load_forgesight from load_data.py, predict the MEAN of y_wear_train for every row in the validation set, and print the resulting MAE and RMSE in microns. Print the standard deviation of y_wear_train alongside them. Add a comment explaining why RMSE for a mean-predictor is close to the target's standard deviation.
```

2. Run it and write the baseline MAE down. Every number you produce for the rest of this lab is judged against it. Expect a baseline MAE in the region of tens of microns. Write the exact number in review_notes.md. Learners who skip this step routinely celebrate an MAE that is worse than predicting the average. COMMAND — run this in your terminal:

```
python baseline_wear.py
```

3. Prompt for the model and the training script. Note that the prompt names the shapes, the loss and the exact reporting format. Two clauses are doing heavy lifting. 'NO activation on the output' — a ReLU there would clamp every prediction to be non-negative, which sounds harmless for wear but silently caps the model; a sigmoid there would squash all predictions into (0,1) and make the task impossible. 'Print both shapes once before the first loss call' — this is the Lab 4 broadcast trap, made visible. PROMPT — paste this into your AI coding assistant:

```
Create models.py and train_wear.py for a PyTorch regression task.
```

```
SHAPES: X_train is float32 (2800, 6), y_wear_train is float32 (2800,). Features are already standardised from load_data.py.
```

```
TASK: predict tool_wear_um, a continuous value in microns.
```

```
OPERATIONS: in models.py define class WearNet(nn.Module) with Linear(6,64) -> ReLU -> Linear(64,32) -> ReLU -> Linear(32,1) and NO activation on the output. In train_wear.py use nn.MSELoss and torch.optim.Adam at lr=1e-3, batch size 64 via TensorDataset and DataLoader, and train for 100 epochs.
```

```
CONSTRAINTS: reshape the target to (N,1) so it matches the prediction shape exactly - print both shapes once before the first loss call to prove they match.
```

Set manual seed 42. Evaluate under `model.eval()` and `torch.no_grad()`. Do not apply any activation to the output layer.

EXPECTED OUTPUT: per-epoch train and validation loss every 10 epochs; final validation MAE and RMSE in microns; a saved plot of both loss curves at `reports/wear_curve.png`.

4. Read the generated code and check three things before running: the output layer has no activation, the target is reshaped, and `eval()` plus `no_grad()` wrap the validation pass. If the printed shapes are `torch.Size([64, 1])` and `torch.Size([64, 1])`, you are safe. If you see `torch.Size([64])` for the target, stop and fix it — the loss will broadcast to (64, 64) and every number after that is meaningless, without any error being raised.

5. Run the training. Watch that both losses fall and that the printed shapes match. Training 100 epochs on 2800 rows takes well under a minute on CPU. If the loss is not falling at all, check the learning rate first; if it explodes to nan, the target was probably not standardised or the learning rate is far too high. COMMAND — run this in your terminal:

```
python train_wear.py
```

6. Compare your final validation MAE against the baseline MAE you wrote down. State the improvement as a percentage. A good result here is a validation MAE meaningfully below the baseline. If the improvement is under a few percent, the features may genuinely not carry much signal about wear — which is a legitimate finding to report, not a failure to hide. Say so out loud; that honesty is the professional habit being trained.

7. Open `reports/wear_curve.png` and read the two curves. Note whether validation loss is still falling at epoch 100 or has flattened. If validation loss is still falling, the model is underfitted and more epochs would help. If it has flattened while training loss keeps dropping, you are watching the beginning of overfitting — the exact pattern you will diagnose properly in Lab 14.

Test it

`baseline_wear.py` prints a baseline MAE and RMSE. `train_wear.py` prints matching prediction and target shapes of `torch.Size([N, 1])`, trains with both losses falling, and reports a final validation MAE clearly BELOW the baseline MAE. `reports/wear_curve.png` shows both curves labelled.

Note: Full commands, prompts and sample code are in `labs/lab08-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 9 — Vibe Coding a Classification Model with Softmax and Cross Entropy

Learning outcome: build a multi-class classifier that outputs raw logits and pair it correctly with cross entropy loss.

Goal: Predict the ForgeSight quality-control class — ok, scratch, dent or burr — from the same six sensors. This lab is deliberately a trap. You first ask the assistant for the model in a way that invites the classic mistake, and there is a strong chance it hands you a network with `nn.Softmax` on the output next to `nn.CrossEntropyLoss`. You catch it, prove numerically that it is wrong, and fix it. You then evaluate properly against a majority-class baseline with a confusion matrix, because on imbalanced classes accuracy alone will flatter a model that never predicts the rare defect at all.

What you'll build

A QC classifier outputting raw logits, evaluated with accuracy, per-class recall and a confusion matrix against the majority baseline (Tools: PyTorch, `nn.CrossEntropyLoss`, scikit-learn metrics, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Establish the majority-class baseline first, and look at how imbalanced the classes actually are. With four classes and a realistic factory distribution, most readings are 'ok'. A majority-class predictor therefore scores far above 25% while being completely useless — it never flags a single defect. This is why accuracy alone cannot be the reported metric and why per-class recall matters. PROMPT — paste this into your AI coding assistant:

```
Create baseline_qc.py. Using load_forgesight, print the count and percentage of each of the 4 qc_class values in the training split. Then predict the single most frequent class for every validation row and print the resulting accuracy. Print the per-class recall for that baseline as well, and add a comment on what recall is for the classes it never predicts.
```

2. Run it. Note the majority-class accuracy — this is the number your model must beat, and it is higher than most people expect. Write the baseline accuracy down. If your trained model ends up close to it, the model is probably just predicting 'ok' for everything — check the confusion matrix rather than celebrating the accuracy. COMMAND — run this in your terminal:

```
python baseline_qc.py
```

3. Now prompt for the classifier, deliberately WITHOUT specifying the output activation. You are setting a trap for the assistant. The prompt names the layers but deliberately says nothing about the output activation. This is exactly how these prompts get written in real life, and it is why the bug is so common. Assistants frequently add a softmax because it 'looks like' what a classifier should do. PROMPT — paste this into your AI coding assistant:

```
Add class QCNet(nn.Module) to models.py and create train_qc.py.
```

SHAPES: X_train is float32 (2800, 6); y_qc_train is int64 (2800,) with 4 classes.

TASK: classify each machine reading into one of 4 quality-control classes.

OPERATIONS: QCNet should be Linear(6,64) -> ReLU -> Linear(64,32) -> ReLU -> Linear(32,4). Train with nn.CrossEntropyLoss and Adam at lr=1e-3, batch size 64, 100 epochs.

CONSTRAINTS: manual seed 42, evaluate under model.eval() and torch.no_grad().

EXPECTED OUTPUT: per-epoch train and validation loss, final validation accuracy, per-class precision and recall, and a confusion matrix saved to reports/qc_confusion.png.

4. STOP before running. Inspect the generated QCNet output layer. Is there an nn.Softmax, nn.LogSoftmax or F.softmax anywhere after the final Linear? Look at the last layer of the Sequential or the end of forward(). Any of nn.Softmax(dim=1), F.softmax(x, dim=1) or nn.LogSoftmax before returning is the bug. If your assistant returned raw logits, it got it right — still run the proof script, because you need to see the numbers.

5. If a softmax is present, prove it is wrong before removing it rather than just deleting it. Expect the double-softmax loss to be noticeably different and its gradients an order of magnitude smaller. Smaller gradients mean smaller updates mean slower learning — the model still trains, the loss still falls, and it simply ends up worse. Nothing warns you. This is the whole lesson. PROMPT — paste this into your AI coding assistant:

Write a short script proof_softmax.py that takes one batch of logits of shape (8, 4) with manual seed 42 and computes nn.CrossEntropyLoss twice: once on the raw logits, and once on torch.softmax(logits, dim=1). Print both loss values and both gradients with respect to the logits, and print the ratio of the gradient magnitudes. Add a comment explaining that CrossEntropyLoss applies log_softmax internally, so the second version applies softmax twice and produces much smaller gradients.

6. Fix the model with a targeted follow-up, then train it. Note the last clause: softmax is not banned, it is relocated. You still want probabilities when reporting a confidence to a user — you just compute them at reporting time, outside the loss path. PROMPT — paste this into your AI coding assistant:

In models.py, QCNet applies softmax to its output, but nn.CrossEntropyLoss already applies log_softmax internally, so the network is applying it twice and its gradients are being flattened. Remove the softmax layer so QCNet returns raw logits from the final Linear layer. Keep everything else unchanged. Where a probability is needed for reporting, apply torch.softmax at that point instead.

7. Train the corrected model and compare accuracy and per-class recall against the majority baseline. A corrected model should beat the majority baseline on accuracy AND find a meaningful share of at least the common defect classes. If accuracy is

high but recall on the rare classes is near zero, say so — that is an honest and important result about class imbalance. **COMMAND** — run this in your terminal:

```
python train_qc.py
```

8. Open `reports/qc_confusion.png` and find which class the model confuses most. Note whether it predicts the rarest class at all. The confusion matrix is the real report. Off-diagonal mass tells you which defects look alike to the model. A completely empty row means the model never predicts that class at all, which no accuracy number would have told you.

Test it

`baseline_qc.py` prints class counts and the majority-class accuracy. `proof_softmax.py` prints two different loss values and shows the double-softmax gradients are much smaller. `train_qc.py`'s QCNet returns raw logits with no softmax layer, and reports a validation accuracy above the majority baseline with a confusion matrix saved to `reports/qc_confusion.png`.

Note: Full commands, prompts and sample code are in `labs/lab09-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 10 — Generating Training Loops, Optimizers and Metrics from Prompts

Learning outcome: refactor training into one reusable fit function and compare optimizers and learning rates with it.

Goal: You have now written the same training loop twice. Refactor it once, properly, and never write it again. You prompt for `engine.py` containing a single `fit()` that takes a model, loaders, a loss, an optimizer and a metric function, returns a history dictionary, and works unchanged for both the regression and the classification task. With that in place, experimentation becomes cheap: you run a small sweep comparing SGD against Adam across three learning rates and produce a results table that shows learning rate matters more than the choice of optimizer.

What you'll build

A reusable `engine.py` `fit/evaluate` pair driving both models, plus a sweep table comparing optimizers and learning rates (Tools: PyTorch, `torch.optim`, `DataLoader`, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Prompt for the reusable engine. The hard requirement is that it must work for BOTH tasks without modification. Point 2 is a real subtlety most generated loops get wrong: averaging the per-batch means is only correct when every batch is the same

size, and the last batch usually is not. Weighting by batch size is the correct reduction. Point 6 is early stopping's useful half — keeping the best weights rather than the last. PROMPT — paste this into your AI coding assistant: ``text Create engine.py with two functions that work unchanged for both a regression and a classification task. fit(model, train_loader, val_loader, loss_fn, optimizer, epochs, metric_fn=None, device='cpu') should:

2. Loop over epochs; for each batch call optimizer.zero_grad() BEFORE the forward pass, compute the loss, call backward, then step
 3. Accumulate the training loss weighted by batch size, not a plain mean of batch means
 4. After each epoch run a validation pass under model.eval() and torch.no_grad(), then return the model to train mode
 5. Apply metric_fn(preds, targets) on the validation set if one is given
 6. Return a history dict with keys train_loss, val_loss and val_metric, each a list of length epochs
 7. Track and restore the state_dict from the epoch with the best validation loss, and report which epoch that was evaluate(model, loader, loss_fn, metric_fn, device) should run one pass under eval/no_grad and return the loss and metric.
- CONSTRAINTS: no printing inside fit except an optional every-N-epochs line controlled by a verbose argument; no task-specific logic - the caller supplies loss_fn and metric_fn. EXPECTED OUTPUT: engine.py importable by both train_wear.py and train_qc.py. ``
8. Review the generated fit() against your Lab 6 checklist before you trust it — this function is about to run every experiment for the rest of the course. Run your checklist: is zero_grad() before backward()? Is the validation pass wrapped in eval() and no_grad()? Does it return to train() afterwards — a loop that forgets this leaves dropout off for the rest of training. Is anything task-specific hiding in there, like an argmax that only makes sense for classification?
 9. Rewire both existing training scripts to use it, and confirm the results still match what you got in Labs 8 and 9. A shared engine is only safe if it is genuinely general. If fit() contains an argmax or a .float() cast that only suits one task, it will silently corrupt the other. Making both scripts use it is the test. PROMPT — paste this into your AI coding assistant:

Refactor train_wear.py and train_qc.py to use fit and evaluate from engine.py instead of their own inline training loops. Keep the same seeds, architectures, hyperparameters and reporting so the results are directly comparable. train_wear.py passes nn.MSELoss and an MAE metric function; train_qc.py passes nn.CrossEntropyLoss and an accuracy metric function. Delete the now-duplicated loop code from both files.

10. Re-run both and check the final numbers are essentially unchanged from the previous labs. A refactor that changes results is a refactor that introduced a bug. 'Essentially unchanged' means within normal run-to-run variation, not bit-identical —

the batching order may differ slightly. If the numbers move a lot, diff the old loop against `fit()` and find what changed. This is a genuine regression test, and it is why you kept the seeds fixed. **COMMAND** — run this in your terminal:

```
python train_wear.py
```

```
python train_qc.py
```

11. Now use the engine for what it was built for. Prompt for a sweep across optimizers and learning rates. Re-instantiating the model each run is the clause that makes the comparison valid. Reusing a trained model across configurations means each run starts from the previous one's weights and the table is nonsense.

Assistants get this wrong regularly — check it explicitly in the generated code.

PROMPT — paste this into your AI coding assistant:

```
Create sweep.py using fit from engine.py.
```

```
TASK: compare optimizers and learning rates on the QC classification task.
```

```
OPERATIONS: for each combination of optimizer in [SGD with momentum 0.9, Adam] and learning rate in [1e-2, 1e-3, 1e-4], train a FRESH QCNet for 60 epochs with manual seed 42 reset before each run, and record the best validation loss, the validation accuracy at that epoch, the epoch it occurred, and the wall-clock seconds.
```

```
CONSTRAINTS: identical data, batch size and seed across all 6 runs - only the optimizer and learning rate change. Re-instantiate the model each run so no weights carry over.
```

```
EXPECTED OUTPUT: a printed table sorted by best validation loss, saved to reports/sweep.csv, plus a single figure overlaying all 6 validation loss curves with a legend.
```

12. Run the sweep and read the table. Answer: does the optimizer or the learning rate account for more of the spread in results? The usual finding: SGD at $1e-4$ barely moves, Adam at $1e-2$ is unstable, and the middle settings work. The spread across learning rates within one optimizer is typically much larger than the spread across optimizers at a sensible learning rate. Learning rate is the hyperparameter worth your attention. **COMMAND** — run this in your terminal:

```
python sweep.py
```

13. Record the winning configuration and your answer in `prompts.md`, and adopt that configuration for the rest of the course. You will reuse `fit()` in Labs 13 to 19 without modification. That is the payoff: from here on, a new model is a new `nn.Module` plus a metric function, not another hand-written loop with another chance to forget `zero_grad()`.

Test it

`engine.py`'s `fit` runs both `train_wear.py` and `train_qc.py` unchanged and reproduces the Labs 8 and 9 results. `sweep.py` produces `reports/sweep.csv` with 6 rows, a sorted

printed table, and an overlay figure of 6 validation curves. You can state which factor - optimizer or learning rate - drove more of the difference.

Note: Full commands, prompts and sample code are in labs/lab10-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 11 — Saving, Loading and Iterating on Models

Learning outcome: save a model's state_dict with its configuration, reload it into a fresh instance and prove the predictions are identical.

Goal: A model that only exists in memory is not a deliverable. You prompt for a checkpoint helper that saves the state_dict together with everything needed to rebuild it — architecture arguments, the standardisation statistics, the class names, the metrics and the library versions — then reload it into a freshly constructed model in a NEW process and assert the predictions match to machine precision. That assertion is the difference between believing your model saved and knowing it did. You finish by writing the first ForgeSight model card.

What you'll build

A checkpoint helper saving weights plus config and statistics, a proven identical-prediction reload, and a model card (Tools: PyTorch, torch.save / torch.load, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Prompt for the checkpoint helper. The key insight is that the file must contain everything needed to rebuild, not just the weights. Pickling the whole model object embeds your file paths and class definitions, so it breaks when the code moves or the class is renamed. A state_dict is just tensors, and the config tells you how to rebuild the shell they go into. This is why the official PyTorch recommendation is state_dict plus code. PROMPT — paste this into your AI coding assistant:

Create checkpoint.py with save_checkpoint and load_checkpoint functions.

save_checkpoint(path, model, config, feat_mean, feat_std, metrics, class_names=None) must save a single dict containing: the model's state_dict; the config dict of constructor arguments needed to rebuild the class; the model class NAME as a string; feat_mean and feat_std tensors; the metrics dict; the class names; the torch version; and an ISO timestamp.

load_checkpoint(path, model_class) must construct a fresh instance from the saved config, load the state_dict into it, call model.eval(), and return the model plus the full metadata dict.

CONSTRAINTS: save the `state_dict`, never the pickled model object - explain why in a comment. Create the `models/` directory if missing. Use `weights_only=False` on load only where required by your torch version, and note why in a comment.

EXPECTED OUTPUT: both functions importable, with a `__main__` block that round-trips a small dummy model as a self-test.

2. Wire it into the QC training script so a completed training run always leaves a loadable artifact behind. `feat_mean` and `feat_std` are part of the model, not part of the training script. If they are not in the checkpoint, whoever loads this model six months from now has no way to preprocess input correctly, and nothing will tell them they got it wrong. PROMPT — paste this into your AI coding assistant:

Update `train_qc.py` to call `save_checkpoint` at the end of training. Save to `models/qc_net_v1.pt` with the config needed to rebuild QCNet, the `feat_mean` and `feat_std` returned by `load_forgesight`, the final validation accuracy and per-class recall as metrics, and `class_names=['ok','scratch','dent','burr']`.

3. Run training so the checkpoint is written. Check the file actually appeared and note its size — a few hundred kilobytes for this network. If it is suspiciously large, the whole model object was pickled rather than the `state_dict`. COMMAND — run this in your terminal:

```
python train_qc.py
```

4. Now prove the reload works, from a SEPARATE script that never sees the trained object in memory. The separate-script constraint is the real test. Loading in the same process as training can pass by accident because the trained object is still in memory. A fresh process proves the file alone is enough. PROMPT — paste this into your AI coding assistant:

Create `reload_check.py` that proves the checkpoint round-trips exactly.

OPERATIONS: (1) load `models/qc_net_v1.pt` with `load_checkpoint`, rebuilding QCNet from the saved config; (2) take the first 16 rows of the validation set; (3) run them through the reloaded model under `torch.no_grad()`; (4) print the predicted class indices and the saved metrics; (5) assert the reloaded model's predictions are identical to the predictions the training script produced for the same rows, which you should also save alongside the checkpoint for this purpose; (6) print the saved timestamp, torch version and class names.

CONSTRAINTS: this script must NOT import or re-run training - it may only read the checkpoint file.

5. Run it and confirm the assertion passes. This is the moment you know the artifact is real. Identical means exactly identical — same architecture, same weights, eval mode, no dropout randomness. If predictions differ, the usual causes are the model left in train mode, or the config rebuilding a different architecture from the one that was trained. COMMAND — run this in your terminal:

```
python reload_check.py
```

6. Deliberately break it to see the failure mode you are guarding against. This is the failure you are being inoculated against. Raw sensor values are orders of magnitude away from the standardised range the network trained on, so the predictions are garbage — but they are still valid class indices with confident probabilities. Nothing crashes. Nothing warns. PROMPT — paste this into your AI coding assistant:

Add a commented-out demonstration to `reload_check.py`: reload the model but skip applying `feat_mean` and `feat_std` to the input rows, so the model receives raw unstandardised sensor values. Print the predictions from both paths side by side and a comment noting that no error is raised - the model simply predicts confidently and wrongly.

7. Write the model card. Do it in your own words — this is the document that travels with the model. A model card states: what it predicts, what data trained it, how it scored (with the baseline for comparison), the known weaknesses from your confusion matrix, and explicitly where it must NOT be used — for example, on a machine type absent from the training data. Keep it to one page.

Test it

`models/qc_net_v1.pt` exists and contains the `state_dict`, `config`, `feat_mean`, `feat_std`, `metrics`, class names, torch version and timestamp. `reload_check.py` runs in a fresh process, rebuilds QCNet from the checkpoint alone, and its assertion of identical predictions passes. `models/qc_model_card.md` states the metric, the baseline and at least one place the model must not be used.

Note: Full commands, prompts and sample code are in `labs/lab11-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Topic 03 — Vibe Coding Convolutional Neural Networks (~24% of course time)

Overview of CNNs: convolution, pooling and padding · Vibe coding a CNN image classifier · Diagnosing overfitting with AI assistance · Data augmentation and regularization via prompts · Transfer learning with pre-trained models

Key concepts

- A convolution slides a small learned kernel across the image, so the same feature detector works wherever the feature appears - this is why CNNs beat dense layers on images.
- Padding preserves spatial size at the border; stride and pooling shrink it. Getting these wrong is how the flatten-to-linear layer ends up with a shape mismatch.

- Pooling downsamples and buys a little translation tolerance. Channels grow as spatial size shrinks: the network trades WHERE for WHAT as it goes deeper.
- Overfitting is the model memorising training images. You see it as a widening gap between training and validation loss, not as a bad training number.
- Data augmentation - random flips, rotations, crops, colour jitter - manufactures new training views for free. Augment the TRAINING set only, never validation.
- Dropout, weight decay and early stopping are the three regularizers you reach for first. Dropout must be switched off at evaluation time by `model.eval()`.
- Transfer learning reuses a network already trained on millions of images: freeze the backbone, replace the classifier head, and fine-tune on a few hundred of your own images.

Lab 12 — Overview of CNNs: Convolution, Pooling and Padding

Learning outcome: reason about convolution, padding, stride and pooling well enough to predict a feature map's shape before running it.

Goal: ForgeSight gains a camera. You generate the surface-inspection image set — 1200 greyscale images of machined parts labelled ok, scratch or dent — then build a shape explorer rather than a model. You apply Conv2d and MaxPool2d with different kernel sizes, paddings and strides, predicting each output shape from the formula before you print it. You visualise what an edge-detecting kernel actually does to a scratch image, and you finish by deriving the flatten size that a classifier's first Linear layer needs — the number that causes more CNN shape errors than anything else.

What you'll build

A generated 1200-image defect dataset plus a `conv_explorer.py` that predicts and verifies every feature-map shape (Tools: PyTorch, torchvision, Pillow, matplotlib, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Generate the ForgeSight surface-inspection images. Copy `make_images.py` from the course resources and run it. `make_images.py` synthesises the dataset deterministically with a fixed seed, so everyone in the room gets identical images. It creates `data/defects/train/{ok,scratch,dent}` and `data/defects/val/{ok,scratch,dent}`, roughly 1200 images at 64x64 greyscale. It takes under a minute and needs no internet. COMMAND — run this in your terminal:

```
python make_images.py
```

2. Inspect what you generated before you model it. Never train on a dataset you have not looked at. Counting per class matters because an imbalanced image set produces the same flattering-accuracy trap you met in Lab 9. `ToTensor()` also rescales pixel values from 0-255 into 0-1 and moves the channel axis to the front, giving (1, 64, 64) — note both changes, because forgetting the rescale is a classic bug. PROMPT — paste this into your AI coding assistant:

Create `peek_images.py` that loads the generated dataset from `data/defects/` and: (1) prints how many images are in each class folder for both train and val; (2) loads one image and prints its size, mode and the shape it becomes as a tensor via `torchvision.transforms.ToTensor()`; (3) saves a 3x4 grid figure to `reports/sample_defects.png` showing four examples of each of the three classes with the class name as the subplot title.

CONSTRAINTS: use PIL and torchvision only. Do not train anything.

3. Run it, open `reports/sample_defects.png`, and describe out loud what visually distinguishes a scratch from a dent. If you cannot see it, the model will struggle too. Scratches are thin, high-contrast, directional lines; dents are softer, rounder, lower-contrast blobs. That difference is precisely what a convolutional kernel is good at picking up, and it is why the edge detector in the last step will light up on scratches. **COMMAND** — run this in your terminal:

```
python peek_images.py
```

4. Now the shape work. Write down the output-size formula and predict three shapes on paper before prompting. The formula is $\text{out} = \text{floor}((W - K + 2P) / S) + 1$, where W is the input size, K the kernel size, P the padding and S the stride. Predict (a), (b) and (d) now: with $W=64$ they come out as 62, 64 and 32. The 'same padding' rule worth memorising is $P = (K-1)/2$ for odd K , which is why 3 pairs with padding 1 and 5 pairs with padding 2.

5. Prompt for the explorer, which forces you to commit to a prediction before it reveals the answer. The assert is what makes this a lab rather than a demo. If an assertion fails, the formula in the script is wrong, not PyTorch — read which configuration failed and recompute by hand. Configurations (b) and (c) both preserve 64x64; that is 'same' padding. **PROMPT** — paste this into your AI coding assistant:

Create `conv_explorer.py` demonstrating how convolution and pooling change tensor shape.

SHAPES: the input is a batch of 8 greyscale images of shape (8, 1, 64, 64).

TASK: show the output shape of each operation and confirm it against the analytic formula.

OPERATIONS: for each of these configurations, print the config, the analytically computed output size using the formula $\text{floor}((W - K + 2P)/S) + 1$, and the ACTUAL shape from running the layer, then assert they agree:

(a) `Conv2d(1, 8, kernel_size=3, padding=0, stride=1)`

(b) `Conv2d(1, 8, kernel_size=3, padding=1, stride=1)`

(c) `Conv2d(1, 8, kernel_size=5, padding=2, stride=1)`

(d) `Conv2d(1, 8, kernel_size=3, padding=1, stride=2)`

(e) `MaxPool2d(kernel_size=2, stride=2)` applied after (b)

Then build a small stack `Conv(1,8,3,p=1) -> ReLU -> MaxPool(2) -> Conv(8,16,3,p=1) -> ReLU -> MaxPool(2)` and print the shape after every single layer, finishing with the number of features a Flatten would produce.

CONSTRAINTS: no training, no gradients, manual seed 42. Print a comment line explaining which configuration preserves spatial size and why.

6. Run it and check your three paper predictions against the printed shapes. The stack should print something like `(8,8,64,64) -> (8,8,32,32) -> (8,16,32,32) -> (8,16,16,16)`, giving a flatten of `161616 = 4096` features. Notice the pattern: channels double as spatial size halves. The network is trading WHERE information for WHAT information as it goes deeper. COMMAND — run this in your terminal:

```
python conv_explorer.py
```

7. Visualise what a convolution actually computes, so the operation stops being abstract. Setting the kernel weights by hand is the point. A learned CNN discovers kernels like these on its own — seeing a hand-built edge detector respond to a scratch makes the first convolutional layer concrete rather than magical. PROMPT — paste this into your AI coding assistant:

Add a section to `conv_explorer.py` that takes one scratch image and one ok image from `data/defects/`, applies three FIXED 3x3 kernels - a horizontal edge detector, a vertical edge detector, and a blur - using `F.conv2d` with manually set weights rather than learned ones, and saves a figure to `reports/feature_maps.png` showing the original next to the three responses for both images. Add a comment on which kernel responds most strongly to a scratch and why.

8. Run it, open the figure, and note which kernel makes the scratch most visible. Record the final flatten size from the stack — you need it in Lab 13. Write the flatten size in `prompts.md`. In Lab 13 the first Linear layer must accept exactly this number, and getting it wrong is the single most common CNN error. Now you can derive it instead of guessing. COMMAND — run this in your terminal:

```
python conv_explorer.py
```

Test it

`data/defects/` contains train and val folders with three class subfolders each and roughly 1200 images total. `conv_explorer.py` prints predicted and actual shapes for all five configurations with every assertion passing, prints the full stack's per-layer shapes ending in a flatten size of 4096, and `reports/feature_maps.png` shows the edge kernels responding to a scratch.

Note: Full commands, prompts and sample code are in `labs/lab12-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 13 — Vibe Coding a CNN Image Classifier

Learning outcome: build, train and evaluate a convolutional network that classifies images into three defect classes.

Goal: Turn the shape knowledge into a working classifier. You prompt for an ImageFolder-based data pipeline and a DefectCNN whose first Linear layer takes exactly the flatten size you derived in Lab 12, then train it with the same engine.fit() you built in Lab 10 — no new training loop. You evaluate against a majority-class baseline with a confusion matrix, and you inspect the images the model gets wrong. Because the architecture, the loss and the loop are all things you have already validated, the only new failure surface is the data pipeline and the shape arithmetic.

What you'll build

A trained DefectCNN classifying ok, scratch and dent, evaluated with a confusion matrix and a misclassified-image grid (Tools: PyTorch, torchvision ImageFolder, engine.py, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Prompt for the image data pipeline. Keep the transforms minimal for now — augmentation is deliberately Lab 15's job. ImageFolder assigns class indices alphabetically, so expect dent=0, ok=1, scratch=2 — not the order you would naturally write. Reading a confusion matrix with the wrong mapping in your head is a classic and entirely avoidable mistake. Normalize with mean 0.5 and std 0.5 maps the 0-1 pixel range to roughly -1 to 1, which is a reasonable default for a network with ReLU activations. PROMPT — paste this into your AI coding assistant:

Create `datasets.py` for the defect images.

SHAPES: `data/defects/train` and `data/defects/val` each contain subfolders `ok`, `scratch` and `dent` holding 64x64 greyscale PNGs.

TASK: return `DataLoaders` for training and validation.

OPERATIONS: build a function `get_defect_loaders(batch_size=32, data_dir='data/defects')` that uses `torchvision.datasets.ImageFolder` with a transform pipeline of `Grayscale(1)` then `ToTensor()` then `Normalize(mean=[0.5], std=[0.5])`. Return `train_loader` (shuffled), `val_loader` (not shuffled) and the `class_to_idx` mapping.

CONSTRAINTS: do NOT add any augmentation - that comes in a later lab. Set a fixed generator seed of 42 for the training shuffle. Print the `class_to_idx` mapping and the number of images in each split when run as a script.

EXPECTED OUTPUT: importable loaders plus a printed summary showing the three classes and their counts.

2. Run it and confirm the class mapping. Note which integer maps to which class name — you will need it to read the confusion matrix. If the counts are badly

imbalanced, note it now — it changes how you must read the accuracy later. The generated set is roughly balanced, so a majority-class baseline should sit near a third. **COMMAND** — run this in your terminal:

```
python datasets.py
```

3. Prompt for the CNN. Give it the flatten size you derived in Lab 12 rather than letting it guess. $6488 = 4096$, which is the number you derived in Lab 12: three `MaxPool2d(2)` layers take 64 to 32 to 16 to 8, with 64 channels at the end. If you let the assistant guess this number it will often be wrong, and the error appears only at the first forward pass. **PROMPT** — paste this into your AI coding assistant:

```
Add class DefectCNN(nn.Module) to models.py.
```

```
SHAPES: input batches are (N, 1, 64, 64); there are 3 output classes.
```

```
ARCHITECTURE: Conv2d(1,16,3,padding=1) -> ReLU -> MaxPool2d(2) ->
Conv2d(16,32,3,padding=1) -> ReLU -> MaxPool2d(2) -> Conv2d(32,64,3,padding=1) ->
ReLU -> MaxPool2d(2) -> Flatten -> Linear(64*8*8, 128) -> ReLU -> Linear(128, 3).
```

```
CONSTRAINTS: the final layer must output RAW LOGITS with no softmax, because
training uses nn.CrossEntropyLoss. Add a comment above the Flatten deriving 64*8*8
from three poolings of a 64x64 input. Include a __main__ block that runs a random
(4,1,64,64) tensor through the model and prints the shape after each block to
prove the arithmetic.
```

4. Before training, run the model's self-test and confirm the printed shapes match your own derivation. The self-test is the cheapest possible verification — a random tensor and a set of printed shapes, no data loading and no training. If the flatten size is wrong you find out in one second rather than after a five-minute training run.

COMMAND — run this in your terminal:

```
python models.py
```

5. Prompt for the training script, reusing the engine rather than writing another loop. Reusing `engine.fit()` is the discipline being tested. If the assistant writes a fresh loop anyway, follow up: 'Use fit from engine.py. Do not write a training loop in this file.'

Every loop you avoid writing is a loop that cannot be missing `zero_grad()`. **PROMPT** — paste this into your AI coding assistant:

```
Create train_cnn.py that trains DefectCNN using fit and evaluate from engine.py -
do NOT write a new training loop.
```

```
OPERATIONS: get the loaders from datasets.py; instantiate DefectCNN with manual
seed 42; train for 25 epochs with nn.CrossEntropyLoss, Adam at lr=1e-3, and an
accuracy metric function; print the majority-class baseline accuracy on the
validation set BEFORE training starts; after training report final validation
accuracy, per-class precision and recall, and save a labelled confusion matrix to
reports/cnn_confusion.png; save a checkpoint to models/defect_cnn_v1.pt using
save_checkpoint from checkpoint.py; save the train and validation loss curves to
reports/cnn_curve.png.
```

CONSTRAINTS: evaluation under `model.eval()` and `torch.no_grad()` - which fit already handles. Use the class names from `class_to_idx` on the confusion matrix axes.

6. Train it. On CPU this takes a few minutes — watch the two loss curves as they print. Expect training accuracy to climb faster than validation accuracy — that widening gap is overfitting starting, and it is exactly what Lab 14 measures. Do not fix it yet. If validation accuracy is stuck near the baseline, check the logits are raw and the learning rate is sensible. COMMAND — run this in your terminal:

```
python train_cnn.py
```

7. Compare final accuracy against the majority baseline, then read the confusion matrix to see which two classes the model confuses. A good result is clearly above the baseline. The confusion the model usually shows is dent against ok, because a faint dent is genuinely low-contrast — which matches what you saw yourself in the sample grid in Lab 12. When your model's mistakes match your own, that is a sign the pipeline is sound.

8. Look at what it got wrong, which tells you more than any single number. Confidently wrong predictions are the interesting ones. If the model is 95% sure an ok part is a dent, look at that image — often there is a real artefact in it, and occasionally you will find a genuinely mislabelled example, which is a normal and important discovery in real datasets. PROMPT — paste this into your AI coding assistant:

Create `inspect_errors.py` that loads `models/defect_cnn_v1.pt`, runs the validation set, selects up to 12 misclassified images, and saves a grid to `reports/cnn_errors.png` where each subplot title shows the true class, the predicted class and the model's confidence in its wrong answer. Sort them by confidence so the most confidently wrong images appear first.

Test it

`datasets.py` prints three classes with their counts and the `class_to_idx` mapping. `models.py`'s self-test prints per-block shapes ending in a flatten of 4096. `train_cnn.py` reports a validation accuracy clearly above the printed majority-class baseline, saves `reports/cnn_confusion.png` with named axes, and writes `models/defect_cnn_v1.pt`. `reports/cnn_errors.png` shows misclassified images with confidences.

Note: Full commands, prompts and sample code are in `labs/lab13-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 14 — Diagnosing Overfitting with AI Assistance

Learning outcome: recognise overfitting from training and validation curves and locate the epoch where generalisation stops improving.

Goal: Your Lab 13 model almost certainly overfits — now measure it rather than guess. You train the same CNN deliberately hard and plot training against validation loss and accuracy on shared axes, then identify the exact epoch where validation stops improving while training keeps falling. To make the pattern unmistakable you also train on a deliberately tiny subset until it reaches near-perfect training accuracy and useless validation accuracy. You finish by asking the assistant to diagnose the curves from a description alone, and you check its reasoning against what you can see.

What you'll build

An overfitting diagnosis with annotated curves, a memorised-subset demonstration, and a written diagnosis you verified yourself (Tools: PyTorch, matplotlib, engine.py, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Train long enough for the problem to appear. Prompt for a diagnostic run with no regularization at all. Sixty epochs with no regularization is not how you would train a production model — it is how you make a phenomenon visible. Marking the minimum-validation-loss epoch on the plot is what turns a vague 'it overfits' into a specific, actionable number. PROMPT — paste this into your AI coding assistant:

```
Create diagnose_overfit.py that trains DefectCNN for 60 epochs using fit from engine.py with NO augmentation, NO dropout and NO weight decay - deliberately unregularized.
```

```
OPERATIONS: record train loss, validation loss, train accuracy and validation accuracy every epoch. Save a 2-panel figure to reports/overfit_curves.png - left panel both losses, right panel both accuracies, epochs on the x axis, with a vertical dashed line marking the epoch of MINIMUM validation loss and that epoch number in the legend. Print: the best validation loss and its epoch; the final training loss; the final validation loss; the final gap between training and validation accuracy in percentage points.
```

```
CONSTRAINTS: manual seed 42, batch size 32, Adam lr=1e-3. Do not stop early - run all 60 epochs so the divergence is visible.
```

2. Run it. This takes several minutes on CPU — while it runs, write down what you expect the two curves to do. Expect validation loss to bottom out somewhere in the first third of training and then climb, while training loss keeps falling towards zero. The rising validation loss is the model becoming more confident about the training images specifically, which is the definition of overfitting. COMMAND — run this in your terminal:

```
python diagnose_overfit.py
```

3. Open reports/overfit_curves.png and answer three questions before reading any explanation: at which epoch does validation loss bottom out, what does training loss do after that, and how wide is the final accuracy gap? The single most important reading: validation loss can rise while validation ACCURACY is still roughly flat. Loss

is sensitive to confidence, accuracy only to the argmax. Loss turns first, which is why it is the better early-stopping signal.

4. Now make the effect undeniable. Prompt for a memorisation demonstration on a tiny subset. Thirty images cannot possibly represent the variation in the full set, so the network simply memorises them. This is the same mechanism as the main run, just fast and obvious. It is also a genuinely useful debugging trick: a model that CANNOT reach high accuracy on 30 images has a bug, not a data problem.

PROMPT — paste this into your AI coding assistant:

```
Add a function memorise_demo() to diagnose_overfit.py.
```

```
TASK: show overfitting in its purest form by training on far too little data.
```

```
OPERATIONS: take only 30 training images (10 per class) using
torch.utils.data.Subset, keep the FULL validation set, and train a fresh DefectCNN
for 100 epochs with the same settings. Plot training and validation accuracy to
reports/memorise.png and print both final accuracies.
```

```
CONSTRAINTS: same seed and architecture as the main run - only the amount of
training data changes.
```

```
EXPECTED OUTPUT: training accuracy approaching 100% while validation accuracy
stays near the majority-class baseline.
```

5. Run it and compare the two numbers. This is memorisation with nothing learned. Expect training accuracy near 100% and validation accuracy near the baseline. Nothing generalised. Keep reports/memorise.png — it is the clearest single picture of overfitting you will produce today. COMMAND — run this in your terminal:

```
python diagnose_overfit.py
```

6. Test the assistant's diagnostic reasoning — and then test the assistant. The numbers in this prompt are illustrative; substitute your own if you prefer. What you are testing is whether the assistant distinguishes fixes that address the CAUSE — more data, augmentation, a smaller model — from those that only limit the damage, like early stopping. Early stopping does not make the model generalise better; it stops you shipping the worse version. PROMPT — paste this into your AI coding assistant:

```
I trained a CNN image classifier for 60 epochs with no regularization. Training
loss fell steadily from 1.05 to 0.04 and training accuracy reached 99%. Validation
loss fell until epoch 14, reaching 0.52, then rose steadily to 0.95 by epoch 60,
while validation accuracy peaked at 78% around epoch 14 and drifted down to 71%.
```

```
Diagnose what is happening, name the epoch I should have stopped at, and rank the
following fixes by how much improvement you would expect for THIS symptom, with a
one-line reason each: more training data, data augmentation, dropout, weight
decay, early stopping, a smaller model, a lower learning rate.
```

```
Be explicit about which of these address the cause and which only limit the
damage.
```

7. Compare the assistant's ranking against your own curves. Does its recommended stopping epoch match the dashed line in your figure? Assistants are generally strong at this diagnosis, which is worth noticing: they are good at pattern-matching a described symptom and weaker at spotting the same problem inside code they just wrote. Use them accordingly — describe symptoms to them, do not ask them to audit themselves.
8. Write the diagnosis into `review_notes.md` in your own words, with the specific numbers from YOUR run. Write your own numbers: best validation loss and its epoch, the final train/validation accuracy gap, and the two fixes you intend to apply in Lab 15. You will compare against these exact figures next lab.

Test it

`reports/overfit_curves.png` shows training loss falling while validation loss turns upward, with a dashed line at the minimum-validation-loss epoch and that epoch printed. `reports/memorise.png` shows near-100% training accuracy against near-baseline validation accuracy on 30 images. `review_notes.md` records your best epoch, your accuracy gap and the fixes you will apply next.

Note: Full commands, prompts and sample code are in `labs/lab14-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 15 — Data Augmentation and Regularization via Prompts

Learning outcome: apply augmentation, dropout, weight decay and early stopping, and measure how much each narrows the overfitting gap.

Goal: Fix what you measured. You add the four standard remedies one at a time — augmentation on the training set only, dropout in the classifier head, weight decay in the optimizer, and early stopping via the best-epoch tracking already in `engine.fit()` — and after each change you record the validation accuracy and the train/validation gap. The critical trap is deliberate: augmentation must never be applied to validation, and an assistant asked for 'augmented loaders' will very often apply it to both, which silently makes your validation score noisy and pessimistic.

What you'll build

A regularized training pipeline plus an ablation table showing the measured contribution of each remedy (Tools: PyTorch, `torchvision.transforms`, `engine.py`, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Add augmented loaders, with an explicit instruction about which split gets augmented. This is the trap. An assistant asked for 'augmented data loaders' will

frequently build one transform and use it for both splits. Augmented validation is not just wrong, it is invisibly wrong: your score changes every run and is systematically pessimistic, so you will conclude your fixes did not work. **PROMPT** — paste this into your AI coding assistant:

```
Update datasets.py with a new function get_augmented_loaders(batch_size=32,
data_dir='data/defects').
```

TASK: return training and validation loaders where **ONLY** the training set is augmented.

OPERATIONS: the **TRAINING** transform is `RandomHorizontalFlip(0.5)`, `RandomRotation(10)`, `RandomResizedCrop(64, scale=(0.8, 1.0))`, then `Grayscale(1)`, `ToTensor()`, `Normalize([0.5],[0.5])`. The **VALIDATION** transform is **ONLY** `Grayscale(1)`, `ToTensor()`, `Normalize([0.5],[0.5])` with **NO** random operations whatsoever.

CONSTRAINTS: this is the critical requirement - validation must be deterministic, because a randomly augmented validation set gives a different score every run and cannot be compared across experiments. Keep the original `get_defect_loaders` unchanged so the two can be compared.

EXPECTED OUTPUT: a `__main__` block that prints the two transform pipelines side by side so the difference is visible, and saves a grid of 8 augmented versions of the **SAME** training image to `reports/augmented_samples.png`.

2. Verify the trap did not catch you. Print both pipelines and confirm no random transform appears in the validation list. Read the two printed pipelines carefully. The validation list must contain exactly three entries — `Grayscale`, `ToTensor`, `Normalize`. If `RandomHorizontalFlip` or `RandomResizedCrop` appears there, fix it before running anything else. **COMMAND** — run this in your terminal:

```
python datasets.py
```

3. Open `reports/augmented_samples.png`. Check the augmentations are plausible — a rotation so extreme that a scratch becomes unrecognisable teaches the model nothing. Augmentation must preserve the label. A horizontal flip of a scratch is still a scratch, so that is safe. If you were classifying digits, a flip would change a 2 into something that is not a 2 — the transformation has to make sense for **YOUR** data, which is a judgement the assistant cannot make for you.

4. Add dropout to the model, in the right place. Dropout belongs in the dense head because convolutional layers already share weights heavily and are naturally regularized; heavy `Dropout2d` early in a CNN tends to hurt. And the `eval()` note matters: dropout left active at evaluation makes your validation score randomly worse, which is the mirror image of the augmented-validation bug. **PROMPT** — paste this into your AI coding assistant:

```
Add class DefectCNNv2(nn.Module) to models.py, identical to DefectCNN but with
nn.Dropout(p=0.3) inserted after the ReLU that follows Linear(4096, 128), and
nn.Dropout2d(p=0.1) after the final MaxPool.
```

Add a comment explaining why dropout goes in the classifier head rather than between early convolutional layers, and a second comment noting that `model.eval()` disables it automatically - which is why evaluation must never run in train mode.

Keep the output as raw logits with no softmax.

5. Run the ablation. This is the actual experiment — five configurations, one change at a time. One change per run is what makes this an ablation rather than a guess. If two things change between rows you cannot attribute the difference, and the table becomes decoration. PROMPT — paste this into your AI coding assistant: ``text Create `train_cnn_v2.py` running an ablation over regularization, using fit from `engine.py`. RUNS - each 40 epochs, fresh model, manual seed 42 reset before each:

6. baseline: DefectCNN, plain loaders, Adam $lr=1e-3$, `weight_decay=0`

7. plus augmentation: DefectCNN, augmented loaders

8. plus dropout: DefectCNNv2, augmented loaders

9. plus weight decay: DefectCNNv2, augmented loaders, `weight_decay=1e-4`

10. plus early stopping: as run 4 but reporting the metrics from the BEST validation epoch rather than the last For each run record: best validation loss and its epoch, final validation accuracy, best validation accuracy, final training accuracy, and the train-validation accuracy gap in percentage points. EXPECTED OUTPUT: a printed table in run order, saved to `reports/ablation.csv`, plus a figure overlaying the five validation loss curves with a legend. ``

11. Run the ablation and read the gap column down the table. Which single change reduced the gap most? Typical finding: augmentation contributes the largest single reduction in the gap, dropout adds a modest further improvement, weight decay a small one, and early stopping does not change the model at all — it changes which epoch's weights you keep. That last distinction is worth stating out loud. COMMAND — run this in your terminal:

```
python train_cnn_v2.py
```

12. Save the best configuration as the new ForgeSight defect model. The regularization field in the metrics is exactly the kind of provenance that makes a checkpoint trustworthy six months later. A model card that records what was tried, not just what was chosen, is far more useful to the next person. PROMPT — paste this into your AI coding assistant:

```
Update train_cnn_v2.py to retrain the winning configuration and save it with save_checkpoint to models/defect_cnn_v2.pt, including in the metrics dict the validation accuracy, the best epoch, and a regularization field listing which techniques were used. Update models/defect_model_card.md with the new score, the baseline for comparison, and one sentence on what changed since v1.
```

13. Compare v2's accuracy and gap against the Lab 14 numbers you wrote in `review_notes.md`. A successful outcome is a smaller train/validation gap AND a validation accuracy at least as good as before. If the gap shrank because training

accuracy collapsed, you over-regularized — reduce the dropout or soften the augmentation.

Test it

`datasets.py` prints a validation pipeline containing no random transforms, and `reports/augmented_samples.png` shows eight plausible variants of one training image. `reports/ablation.csv` holds five rows with a train-validation gap column that narrows down the table. `models/defect_cnn_v2.pt` is saved with a regularization field, and v2's gap is smaller than the Lab 14 gap.

Note: Full commands, prompts and sample code are in `labs/lab15-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 16 — Transfer Learning with Pre-Trained Models

Learning outcome: fine-tune a pre-trained network by freezing its backbone and replacing the classifier head.

Goal: Stop training from scratch. You load a ResNet-18 pre-trained on ImageNet, adapt it to greyscale 64x64 inspection images, freeze the backbone so only a new three-class head learns, and fine-tune. You then unfreeze the last block at a much lower learning rate and compare all three approaches — scratch CNN, frozen backbone, partial fine-tune — on accuracy, training time and trainable parameter count. The lab makes concrete why transfer learning is the default professional starting point when you have hundreds of images rather than hundreds of thousands.

What you'll build

A fine-tuned ResNet-18 defect classifier plus a three-way comparison against your scratch CNN (Tools: PyTorch, torchvision.models, engine.py, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Prompt for the transfer setup, being explicit about the two adaptations a greyscale 64x64 input requires. There are two legitimate ways to feed greyscale 64x64 images to a ResNet: repeat the single channel three times and resize to 224x224 to match ImageNet exactly, or surgically replace the first conv layer to accept one channel. The first reuses the pre-trained weights fully and is the right default; the second discards the first layer's learned filters. Making the assistant state which it chose forces the decision into the open. PROMPT — paste this into your AI coding assistant:

Create `transfer.py` that fine-tunes a pre-trained ResNet-18 on the defect images.

TASK: adapt an ImageNet ResNet-18 to 3-class greyscale 64x64 defect classification.

OPERATIONS: (1) load `torchvision.models.resnet18` with the default pre-trained weights; (2) adapt the input - the simplest correct approach is a transform pipeline that converts the greyscale image to 3 channels and resizes to 224x224 using the ImageNet normalisation statistics, so state clearly in a comment which approach you used and why; (3) FREEZE every parameter by setting `requires_grad=False`; (4) replace `model.fc` with a new `nn.Linear(512, 3)` whose parameters are trainable; (5) print the total parameter count and the TRAINABLE parameter count so the difference is obvious; (6) train for 12 epochs with Adam `lr=1e-3` using `fit` from `engine.py`, passing only the trainable parameters to the optimizer.

CONSTRAINTS: the new head outputs raw logits. Use the augmented training transform and the deterministic validation transform from Lab 15, adjusted for 3-channel 224x224 ImageNet input.

EXPECTED OUTPUT: printed parameter counts, per-epoch losses, final validation accuracy, and the elapsed training time.

2. Before training, check the parameter counts. The trainable number should be a tiny fraction of the total. ResNet-18 has around 11 million parameters; the new head has about 1,500. Training under 0.02% of the network is why this runs quickly and why it does not overfit 1200 images. If the trainable count is in the millions, the freeze did not take — check `requires_grad` was set before `fc` was replaced. **COMMAND** — run this in your terminal:

```
python transfer.py
```

3. Note the accuracy and time, then compare against your Lab 15 scratch model trained for far longer. Expect the frozen backbone to reach comparable or better accuracy than your scratch CNN in a fraction of the epochs, though each epoch is slower because 224x224 inputs are twelve times larger than 64x64. That trade-off is exactly what the comparison table is for.
4. Now unfreeze the last block and fine-tune at a much lower learning rate. The differential learning rate is the professional detail. The pre-trained features took an enormous amount of compute to learn; a `1e-3` update would wreck them in a single epoch. Small learning rate for pre-trained weights, larger for the randomly initialised head, is the rule. **PROMPT** — paste this into your AI coding assistant:

Add a function `partial_finetune()` to `transfer.py`.

OPERATIONS: start from the frozen model trained above, then set `requires_grad=True` for `layer4` and `fc` only. Use two parameter groups in Adam - `layer4` at `lr=1e-4` and `fc` at `lr=1e-3` - and train for a further 8 epochs. Print the new trainable parameter count and the final validation accuracy.

Add a comment explaining why the unfrozen backbone layer uses a learning rate roughly ten times smaller than the head: the pre-trained weights are already good, and a large update would destroy the features you are trying to reuse.

CONSTRAINTS: do not re-initialise the head - continue from the weights just trained.

5. Run the partial fine-tune and record whether the extra 8 epochs improved on the frozen result. Partial fine-tuning sometimes helps meaningfully and sometimes barely moves the number, particularly when your images look nothing like ImageNet photographs. Synthetic greyscale inspection images are quite far from ImageNet, so a modest gain is a perfectly honest result to report. **COMMAND** — run this in your terminal:

```
python transfer.py
```

6. Build the three-way comparison table that answers the practical question. The 'accuracy per minute' column is the one that changes decisions. In a factory with no GPU, an approach that reaches 88% in four minutes usually beats one that reaches 90% in forty. Say which trade-off you are making rather than just picking the top accuracy. **PROMPT** — paste this into your AI coding assistant:

Add a comparison to transfer.py that produces reports/transfer_comparison.csv with one row per approach: (1) DefectCNNv2 trained from scratch with the Lab 15 best configuration; (2) ResNet-18 frozen backbone with a new head; (3) ResNet-18 with layer4 unfrozen. Columns: approach, trainable parameters, epochs trained, wall-clock training seconds, best validation accuracy, and validation accuracy per minute of training. Print the table sorted by best validation accuracy and add a one-line printed conclusion naming which approach you would choose for a factory with 1200 labelled images and no GPU.

7. Save the best transfer model with a checkpoint and a model card entry. Recording which layers were trainable is essential provenance — 'ResNet-18 fine-tuned' is not reproducible, whereas 'ResNet-18, layer4 and fc trainable, lr 1e-4 and 1e-3, 20 epochs total' is. **PROMPT** — paste this into your AI coding assistant:

Update transfer.py to save the best-performing transfer model with save_checkpoint to models/defect_resnet_v1.pt, recording in the metrics dict the approach used, which layers were trainable, the validation accuracy and the training time. Append a section to models/defect_model_card.md comparing the scratch CNN and the transfer model, and stating which one you recommend for deployment and why.

8. Read the comparison table and state your recommendation out loud, with the trade-off you are accepting. There is no single right answer here, and the trainer will push you to defend yours. Model size, inference speed on the plant's hardware, and the cost of a missed defect all belong in the argument alongside accuracy.

Test it

transfer.py prints a trainable parameter count that is a tiny fraction of the ~11M total, trains a frozen-backbone ResNet-18 to a validation accuracy comparable with or better than your scratch CNN, and reports the partial fine-tune result. reports/transfer_comparison.csv holds three rows with parameters, time and accuracy. models/defect_resnet_v1.pt is saved and the model card names a recommendation.

Note: Full commands, prompts and sample code are in labs/lab16-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Topic 04 — Vibe Coding Recurrent Networks for Sequence Data (~23% of course time)

Overview of RNNs, LSTM and GRU · Vibe coding an LSTM for time series forecasting
· Tuning sequence models with follow-up prompts · Evaluating and visualizing model performance · Packaging a complete deep learning project

Key concepts

- A recurrent network carries a hidden state from one timestep to the next, so order matters. A plain RNN forgets quickly because gradients vanish across long sequences.
- An LSTM adds a cell state and three gates - forget, input, output - so it can retain information over hundreds of steps. A GRU merges the gates into a cheaper, faster cell.
- Sequence data must be windowed before a model sees it: turn one long series into many (lookback -> horizon) pairs, and split by TIME, never at random.
- Scale the series using statistics from the training window only. Fitting the scaler on the whole series leaks the future into the past and inflates every score.
- `nn.LSTM` returns output and `(h_n, c_n)`, and `batch_first` changes the axis order. Reading the wrong axis is the classic silent sequence bug - it runs and predicts noise.
- A forecast is judged against a naive baseline: predicting 'tomorrow equals today' is often strong. A model that cannot beat persistence has not learned the series.
- Packaging is what makes the work real: a clear folder structure, pinned dependencies, a README somebody else can follow, and a model card recording where the model must NOT be used.

Lab 17 — Overview of RNNs, LSTM and GRU

Learning outcome: compare RNN, LSTM and GRU cells by their shapes, gates and parameter counts, and demonstrate why plain RNNs forget.

Goal: Sequence models introduce a new axis and a new class of silent bug, so you explore the cells before you forecast with them. You prompt for a script that runs the same input batch through `nn.RNN`, `nn.LSTM` and `nn.GRU`, printing every output and hidden-state shape, the parameter count of each, and what LSTM returns that the others do not. You then demonstrate the vanishing-gradient problem directly: propagate a gradient back through a long sequence in an RNN and in an LSTM and compare the

magnitudes at the first timestep. Finally you meet `batch_first`, the axis-order flag that silently trains models on nonsense.

What you'll build

An `rnn_cells.py` comparing the three cells' shapes, gates and parameter counts, plus a measured vanishing-gradient demonstration (Tools: PyTorch `nn.RNN` / `nn.LSTM` / `nn.GRU`, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Prompt for the three-cell comparison. Insist on printing every returned object's shape, because that is where the confusion lives. The shapes: output is (16, 50, 32) — the hidden state at EVERY timestep — while `h_n` is (1, 16, 32), the final hidden state only, with the leading 1 being `num_layers`. For forecasting you usually want the last timestep of output, which is `output[:, -1, :]`, or equivalently the squeezed `h_n`. Taking `output[:, 0, :]` by mistake gives you the state after ONE timestep — a right-shaped, useless tensor. PROMPT — paste this into your AI coding assistant:

```
Create rnn_cells.py comparing PyTorch's three recurrent cells.
```

```
SHAPES: the input is a batch of 16 sequences, each 50 timesteps long with 1
feature per step, so shape (16, 50, 1) with batch_first=True.
```

```
TASK: show exactly what each cell returns and how many parameters it has.
```

```
OPERATIONS: build nn.RNN, nn.LSTM and nn.GRU each with input_size=1,
hidden_size=32, num_layers=1, batch_first=True. For each: run the batch through
it; print the shape of EVERY returned object, naming them (output, and h_n, and
for LSTM also c_n); print the total parameter count; and print a one-line
description of its gates - RNN has none, GRU has reset and update, LSTM has
forget, input and output.
```

```
Then print a comparison table of the three parameter counts and add a comment
explaining why LSTM has roughly four times the parameters of RNN and GRU roughly
three times.
```

```
CONSTRAINTS: manual seed 42, no training. Label every printed line clearly.
```

2. Before running, predict two shapes: what shape is output and what shape is `h_n`? They are different, and confusing them is the classic bug. Parameter counts follow directly from the gates. An RNN has one weight set; a GRU has three (reset, update, candidate); an LSTM has four (forget, input, output, candidate). That is the whole explanation for the roughly 1:3:4 ratio you will see printed.
3. Run it and check your predictions. If your prediction of `h_n` missed the leading `num_layers` axis, note it. That leading 1 is the reason so much sequence code contains a `.squeeze(0)` whose purpose nobody remembers. COMMAND — run this in your terminal:

```
python rnn_cells.py
```

4. Now demonstrate the `batch_first` trap, which is the reason sequence models silently learn nothing. `batch_first=False` is the PyTorch DEFAULT, which is why this trap is so common: an assistant that omits the flag gives you a model expecting (seq, batch, feature) while your data is (batch, seq, feature). Nothing errors as long as the numbers happen to be compatible, and the model trains on transposed nonsense.

PROMPT — paste this into your AI coding assistant:

```
Add a function batch_first_trap() to rnn_cells.py.
```

```
OPERATIONS: take the same (16, 50, 1) input. Run it through an nn.LSTM built with batch_first=True and one built with batch_first=False, WITHOUT transposing the input for the second. Print the output shape from each and the shape of h_n from each.
```

```
Add a comment explaining what the second model actually did: it interpreted the batch axis as the time axis, so it treated 16 timesteps of a 50-sequence batch. Note that this raises NO error and produces a plausibly shaped tensor, which is why it is so dangerous.
```

```
Then show the correct fix - transposing the input to (50, 16, 1) for the batch_first=False model - and confirm the output shapes now correspond.
```

5. Run it and study what happened. Confirm for yourself that nothing raised an error. The lesson to carry forward: always pass `batch_first` explicitly, and always print the output shape of the first forward pass. Add that to your review checklist alongside the `zero_grad` and `softmax` checks. COMMAND — run this in your terminal:

```
python rnn_cells.py
```

6. Measure the vanishing gradient rather than just describing it. Summing the last timestep and backpropagating to the input is a direct measurement of how far influence reaches back through time. The log y axis is necessary because the decay is exponential — on a linear axis the RNN's early timesteps are indistinguishable from zero. PROMPT — paste this into your AI coding assistant:

```
Add a function vanishing_gradient() to rnn_cells.py.
```

```
TASK: measure how gradient magnitude decays back through time in an RNN versus an LSTM.
```

```
OPERATIONS: create an input of shape (1, 100, 1) with requires_grad=True. Pass it through a single-layer nn.RNN(1, 16, batch_first=True), take the LAST timestep of the output, sum it, and call backward(). Record the absolute gradient magnitude at the input for each of the 100 timesteps. Repeat with an nn.LSTM of the same size.
```

```
Plot both gradient-magnitude curves against timestep on a LOG y axis and save to reports/gradient_flow.png. Print the ratio of gradient magnitude at timestep 0 to timestep 99 for each cell.
```

```
CONSTRAINTS: same seed and hidden size for both so the comparison is fair.
```

7. Run it, open `reports/gradient_flow.png`, and read the two ratios. This is the vanishing gradient, measured. Expect the RNN's gradient at timestep 0 to be orders of magnitude smaller than at timestep 99, while the LSTM's decays far more gently. That gap is precisely what the cell state and the forget gate buy you, and it is why LSTM and GRU replaced plain RNNs for anything longer than a few dozen steps.

COMMAND — run this in your terminal:

```
python rnn_cells.py
```

8. Write your conclusion in `prompts.md`: which cell you will use in Lab 18, and the one-sentence reason. The expected conclusion is LSTM or GRU, because the vibration series in Lab 18 uses a lookback well beyond the range where a plain RNN retains gradient. State it in your own words with the ratio you measured.

Test it

`rnn_cells.py` prints output as `torch.Size([16, 50, 32])` and `h_n` as `torch.Size([1, 16, 32])` for all three cells, plus `c_n` for the LSTM only, with parameter counts in roughly a 1:3:4 ratio. `batch_first_trap` shows a wrongly shaped-but-unraised result and its fix. `reports/gradient_flow.png` shows the RNN gradient decaying far faster than the LSTM's on a log axis.

Note: Full commands, prompts and sample code are in `labs/lab17-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 18 — Vibe Coding an LSTM for Time Series Forecasting

Learning outcome: window and split a time series correctly and train an LSTM that beats a persistence baseline.

Goal: Forecast ForgeSight's vibration sensor. This lab contains two traps that no error message will ever reveal. First, sequence data must be split by TIME — a random split lets the model train on data from after the period it is tested on. Second, the scaler must be fitted on the training window only. You establish the persistence baseline before modelling, prompt for the windowing and the LSTM with both constraints stated explicitly, then verify the split and the scaler yourself before trusting any score. A forecaster that cannot beat 'tomorrow equals today' has not learned the series.

What you'll build

An LSTM vibration forecaster with a time-ordered split and train-only scaling, measured against a persistence baseline (Tools: PyTorch `nn.LSTM`, `pandas`, `engine.py`, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Look at the series before modelling it, and establish the persistence baseline. Persistence is a genuinely strong baseline for smooth physical signals. Vibration does not jump randomly hour to hour, so 'the same as last hour' is often quite accurate. Beating it requires the model to learn actual structure — trend, cycles, drift — rather than just tracking the level. PROMPT — paste this into your AI coding assistant:

Create `baseline_series.py` for `data/vibration_series.csv`, which has a timestamp column and a `vibration_rms` column of roughly 3000 hourly readings.

OPERATIONS: (1) load it, print the row count, the date range and basic statistics; (2) plot the full series to `reports/series_overview.png`; (3) compute the PERSISTENCE baseline on the last 20% of the series - predicting that each value equals the previous value - and print its MAE and RMSE; (4) print the standard deviation of the series for comparison.

Add a comment explaining why persistence is the correct baseline for a forecasting task and what it means if a model cannot beat it.

2. Run it, open the plot, and write the persistence MAE down. This is the number your LSTM must beat. Write the persistence MAE in `review_notes.md` next to your Lab 8 and Lab 14 numbers. If your LSTM's MAE is higher than this, the honest report is that the model adds nothing, however good the plot looks. COMMAND — run this in your terminal:

`python baseline_series.py`

3. Prompt for the windowing, with the time-split constraint stated as the primary requirement. Requirement 4 is subtle and often missed: if you window the full series first and then split, windows near the boundary contain values from both sides, so training windows include validation timesteps. Split first, then window each segment independently. PROMPT — paste this into your AI coding assistant: ``text Create windowing.py that prepares `data/vibration_series.csv` for an LSTM. TASK: turn one long univariate series into supervised (lookback -> horizon) training pairs.

OPERATIONS: build `make_windows(series, lookback=48, horizon=1)` returning `X` of shape `(N, lookback, 1)` and `y` of shape `(N, 1)`. Then build `get_series_loaders(lookback=48, batch_size=64)` that:

4. Loads the series in TIME ORDER - never sorted or shuffled before splitting
5. Splits chronologically into the first 70% train, next 15% validation, final 15% test - by INDEX, not randomly
6. Fits the scaler using the TRAINING SEGMENT ONLY, computing mean and std from the training values, then applies those same values to validation and test
7. Windows each segment separately so no window ever spans a split boundary
8. Returns loaders plus the scaler statistics so predictions can be converted back to the original units CONSTRAINTS: this is critical - do NOT use `train_test_split` with shuffle, and do NOT fit the scaler on the full series. Shuffling is allowed WITHIN the training loader only. Print the index ranges of the three splits and the training mean

and std when run as a script. EXPECTED OUTPUT: printed split ranges proving they are contiguous and non-overlapping, plus X and y shapes for each split. ``

9. Verify the two traps yourself. Check the printed split ranges are contiguous and in ascending order, and find the exact line where the scaler statistics are computed. Read the printed ranges: they must look like 0-2099, 2100-2549, 2550-2999, contiguous and ascending. If the indices are scattered, a shuffle happened. And find the scaler line — it must compute from the training segment. If it uses the whole series, the future has leaked into the past. COMMAND — run this in your terminal:

```
python windowing.py
```

10. Prompt for the model and training, reusing the engine again. `output[:, -1, :]` takes the hidden state after the model has seen all 48 timesteps, which is what you want for a forecast. `output[:, 0, :]` is the state after ONE timestep — same shape, and the model would be forecasting from a single reading while appearing to work perfectly. PROMPT — paste this into your AI coding assistant:

```
Create lstm_forecast.py.
```

```
SHAPES: batches are X of (64, 48, 1) and y of (64, 1).
```

```
OPERATIONS: add class VibrationLSTM(nn.Module) to models.py with
nn.LSTM(input_size=1, hidden_size=64, num_layers=2, batch_first=True, dropout=0.2)
followed by nn.Linear(64, 1). In forward, take the LAST timestep of the LSTM
output with output[:, -1, :] and pass it to the Linear layer - add a comment on
why this is the right slice and what output[:, 0, :] would give instead.
```

```
Train with fit from engine.py: nn.MSELoss, Adam lr=1e-3, 60 epochs, an MAE metric
computed in the ORIGINAL units by inverting the scaling.
```

```
CONSTRAINTS: batch_first=True must be explicit. Print the output shape of the
first forward pass to prove the axis order.
```

```
EXPECTED OUTPUT: per-epoch losses, final test MAE and RMSE in original units,
printed next to the persistence baseline MAE, and a forecast-versus-actual plot of
the test segment saved to reports/forecast.png.
```

11. Train it and compare the MAE against your persistence baseline. A meaningful improvement over persistence is a genuine success here. A small improvement is a common and honest outcome on a smooth series. If your MAE is dramatically better than persistence, be suspicious before being pleased — re-check the split and the scaler first. COMMAND — run this in your terminal:

```
python lstm_forecast.py
```

12. Open `reports/forecast.png` and look at the shape of the prediction, not just the metric. Look for a specific failure mode: a forecast that is simply the previous value shifted right by one step. That is the model learning persistence and nothing more. It looks excellent on a plot and is worth spotting — compare the curve's turning points against the actual series to see whether it anticipates or merely follows.

13. Prove to yourself that the time split mattered, by deliberately doing it wrong. Expect the leaky pipeline to report a noticeably better MAE than the correct one. That number is the one you would have proudly presented, and it is unobtainable in production because you cannot scale today's reading using next month's statistics. This is the time-series version of the Lab 6 lesson. **PROMPT** — paste this into your AI coding assistant:

```
Add a function leaky_comparison() to lstm_forecast.py that repeats the entire pipeline with two deliberate errors: a RANDOM shuffled split instead of a chronological one, and a scaler fitted on the FULL series before splitting. Train the identical model for the same 60 epochs and print the resulting test MAE next to the correct pipeline's MAE and the persistence baseline. Add a comment stating which number you would have reported if you had never checked the split, and why it is not achievable in production.
```

Test it

windowing.py prints three contiguous ascending index ranges and the scaler statistics computed from the training segment only. lstm_forecast.py prints the first forward pass shape as torch.Size([64, 1]), reports a test MAE in original units alongside the persistence baseline, and saves reports/forecast.png. leaky_comparison prints a better-but-unachievable MAE from the shuffled, fully-scaled pipeline.

Note: Full commands, prompts and sample code are in labs/lab18-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 19 — Tuning Sequence Models with Follow-Up Prompts

Learning outcome: improve a sequence model through disciplined follow-up prompts and record what each change actually bought.

Goal: Tuning is where vibe coding either pays off or descends into random prompting. You run a structured search over the choices that matter for a sequence model — lookback length, hidden size, layer count, cell type and learning rate — changing one thing at a time and recording every result in a table. The discipline being trained is the follow-up prompt: naming a specific symptom and a specific change rather than asking the assistant to 'make it better'. You finish with a tuned model, a results table that justifies it, and an explicit note of which changes made no difference at all.

What you'll build

A tuning results table across lookback, hidden size, layers, cell type and learning rate, plus a justified best model (Tools: PyTorch, engine.py, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. First, see what an undisciplined prompt produces, so the contrast is concrete. Expect the assistant to return a model with a different hidden size, an extra layer, a new learning rate, bidirectionality and possibly attention — all at once. If it improves, you cannot say why; if it gets worse, you cannot say why either. This is the failure mode the rest of the lab prevents. PROMPT — paste this into your AI coding assistant:

```
My LSTM forecaster gets a test MAE only slightly better than a persistence baseline. Make it better.
```

2. Read what came back and count how many things it changed simultaneously. Do not run it — this is evidence, not code. Count the changes. Four or five is typical. That is not tuning, it is replacement — and it is exactly what 'make it better' asks for. The prompt was the problem, not the assistant.

3. Now the disciplined version. Prompt for a structured one-factor-at-a-time search. One factor at a time is not the most sample-efficient search that exists, but it is the one that produces an explanation. Keeping the persistence baseline as a constant column is what stops a table of tiny differences from looking like progress when nothing beats the baseline. PROMPT — paste this into your AI coding assistant:

```
Create tune_lstm.py running a one-factor-at-a-time search over the vibration forecasting model.
```

```
BASELINE CONFIG: lookback=48, hidden_size=64, num_layers=2, cell=LSTM, lr=1e-3, 40 epochs.
```

```
RUNS: starting from that baseline, vary ONE factor at a time:
```

```
- lookback in [12, 24, 48, 96]
- hidden_size in [16, 32, 64, 128]
- num_layers in [1, 2, 3]
- cell in [LSTM, GRU]
- lr in [1e-2, 1e-3, 1e-4]
```

```
For each run record: the factor changed, its value, best validation MAE in original units, test MAE in original units, epochs to best, and wall-clock seconds.
```

```
CONSTRAINTS: reset manual seed 42 before every run and build a fresh model each time. Identical data and batch size across all runs. Include the persistence baseline MAE as a constant column so every row can be read against it.
```

```
EXPECTED OUTPUT: a printed table grouped by factor, saved to reports/tuning_results.csv, and a figure with one subplot per factor showing test MAE against the factor's value.
```

4. Run the search. It is a few dozen short training runs — expect several minutes on CPU. Fresh model and reset seed per run is the clause assistants most often drop. If

run 2 continues from run 1's weights the whole table is meaningless — check the generated code for a model built inside the loop, not outside it. **COMMAND** — run this in your terminal:

```
python tune_lstm.py
```

5. Read the table by factor and answer: which factor produced the largest spread in test MAE, and which produced almost none? Typical findings: lookback and learning rate produce the largest spread; hidden size shows diminishing returns past a point; num_layers beyond 2 usually adds time without accuracy; LSTM and GRU come out very close, with GRU faster. Knowing which knobs do nothing is as valuable as knowing which ones work.

6. Practise the disciplined follow-up on whatever your table actually showed. Adapt this to your own result. Notice the structure of this follow-up: it names the file, states the observed pattern with numbers, offers an interpretation, specifies the exact change, and explicitly fences everything else with 'change nothing else'. That fence is what keeps your tuning table comparable across rounds. **PROMPT** — paste this into your AI coding assistant:

```
In tune_lstm.py the lookback sweep shows test MAE improving from 12 to 48
timesteps but getting worse at 96, which suggests the longer window adds noise
rather than signal. Add two intermediate values, 64 and 80, to the lookback sweep
only. Change nothing else - same seed, same architecture, same number of epochs -
and print the extended lookback results as a single sorted table so I can see
where the minimum actually sits.
```

7. Train the winning configuration properly and check it against the baseline one final time. The winning configuration deserves a longer, properly early-stopped run — the sweep used 40 epochs to keep the search fast, which usually undertrains.

Recording the full configuration in the checkpoint means the result is reproducible rather than remembered. **PROMPT** — paste this into your AI coding assistant:

```
Add a function train_best() to tune_lstm.py that takes the best configuration from
reports/tuning_results.csv, trains it for 120 epochs with early stopping on
validation loss, reports test MAE and RMSE in original units next to the
persistence baseline, saves the model with save_checkpoint to
models/vibration_lstm_v1.pt recording the full winning configuration in the
metrics dict, and writes models/vibration_model_card.md stating the task, the
lookback, the baseline comparison and the fact that the split was chronological.
```

8. Record in prompts.md the three follow-up prompts that worked and, just as importantly, which tuned factors turned out not to matter. Writing down what did NOT help is the part everyone skips and the part most worth having. Next time you tune a sequence model you will start from lookback and learning rate instead of adding layers.

Test it

reports/tuning_results.csv contains one row per configuration with the factor changed, both MAE columns and the persistence baseline as a constant column. You can name

the factor with the largest spread and the one with the smallest.
models/vibration_lstm_v1.pt holds the winning configuration and beats the persistence baseline, and prompts.md records three working follow-up prompts.

Note: Full commands, prompts and sample code are in labs/lab19-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 20 — Evaluating and Visualizing Model Performance

Learning outcome: build the evaluation views that reveal what a single metric hides, across all three ForgeSight models.

Goal: One number cannot describe a model. You build the standard evaluation views for each of the three ForgeSight models — a parity plot and residual analysis for tool-wear regression, a normalised confusion matrix with per-class recall for defect classification, and forecast-versus-actual with error-by-horizon for vibration. Each view is chosen to answer a question the headline metric cannot: where is the model biased, which classes does it confuse, and how fast does the forecast decay. You assemble them into one report figure and write the honest summary that goes with it.

What you'll build

A complete evaluation report covering all three models, with the plots and the written findings a stakeholder would need (Tools: PyTorch, matplotlib, scikit-learn metrics, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Build the regression evaluation views for the tool-wear model. A parity plot shows immediately what R-squared summarises away. Points hugging the $y=x$ line across the whole range is a good model; points that flatten towards the mean at the extremes is a model that is hedging — accurate on average and useless where it matters, which is usually at high wear. PROMPT — paste this into your AI coding assistant:

Create evaluate.py with a function eval_regression() for the tool-wear model.

OPERATIONS: load models/wear_net_v1.pt (or retrain briefly if it was not saved), predict on the test split, and produce a 3-panel figure saved to reports/eval_wear.png:

- (1) a parity plot of predicted against actual with a $y=x$ reference line and the R-squared value in the title;
- (2) a residual plot of (predicted - actual) against actual, with a horizontal zero line;

(3) a histogram of residuals with the mean and standard deviation printed in the title.

Also print test MAE, RMSE and R-squared next to the mean-predictor baseline.

Add a comment on what a residual plot that fans out or trends with the actual value tells you about the model.

2. Run it and read panel 2 specifically. Are the residuals centred on zero across the whole range, or does the model drift at high wear values? Residuals that trend with the actual value mean systematic bias, not noise. Residuals that fan out mean the model is reliable for low values and unreliable for high ones. Either is a finding worth reporting; neither is visible in the MAE. **COMMAND** — run this in your terminal:

```
python evaluate.py
```

3. Build the classification evaluation views. Row normalisation answers 'of the parts that really were scratches, what share did we catch?' — which is the question a factory actually asks. Raw counts make a rare class look fine simply because there are few of them. And note where softmax appears: at reporting time, never in the model, exactly as Lab 9 established. **PROMPT** — paste this into your AI coding assistant:

Add `eval_classification()` to `evaluate.py` for the defect CNN.

OPERATIONS: load the best defect model, predict on the validation set, and produce a 3-panel figure saved to `reports/eval_defects.png`:

(1) a confusion matrix normalised BY ROW so each cell shows the share of that true class, with class names on both axes and values annotated;

(2) a horizontal bar chart of per-class precision, recall and F1;

(3) a histogram of the model's maximum softmax probability, split into correct and incorrect predictions.

Print the classification report and the macro-averaged F1 next to the majority-class baseline accuracy.

Add a comment explaining why a row-normalised confusion matrix is more readable than raw counts when classes are imbalanced, and note that softmax is applied HERE for reporting, not in the model.

4. Run it and read panel 3. Is the model well calibrated — are its wrong answers given with lower confidence than its right ones? A well-calibrated model is wrong with low confidence. If the two histograms in panel 3 overlap heavily, the model is confidently wrong a lot of the time, which matters enormously if anyone downstream plans to trust a confidence threshold to decide when to involve a human inspector. **COMMAND** — run this in your terminal:

```
python evaluate.py
```

5. Build the forecasting evaluation views, including the check for the model that only learned persistence. Panel 3 is the panel that matters. A forecaster that has only learned persistence produces a predicted-change scatter that is essentially a flat cloud around zero — it never anticipates a move. A model that genuinely learned structure shows positive correlation between predicted and actual change. **PROMPT** — paste this into your AI coding assistant:

Add `eval_forecast()` to `evaluate.py` for the vibration LSTM.

OPERATIONS: load `models/vibration_lstm_v1.pt`, predict on the test segment, and produce a 3-panel figure saved to `reports/eval_forecast.png`:

- (1) forecast against actual over the test period in ORIGINAL units, with the persistence baseline as a third line;
- (2) a residual-over-time plot showing whether errors grow towards the end of the period;
- (3) a scatter of predicted change against actual change from one timestep to the next - this reveals whether the model is anticipating movement or merely repeating the last value.

Print test MAE and RMSE against the persistence baseline, plus the correlation between predicted and actual CHANGE.

Add a comment explaining that a model whose predicted change correlates near zero with actual change has learned persistence and nothing more.

6. Run it and read panel 3 and the change-correlation figure. This is the honest test of whether your forecaster learned anything. Do not be discouraged by a modest change-correlation. Anticipating short-term movement in a noisy physical signal is genuinely hard, and reporting a small honest number with the plot that proves it is far more professional than reporting an MAE that hides it. **COMMAND** — run this in your terminal:

`python evaluate.py`

7. Assemble the single report figure and write the findings. The report figure is the artifact people will actually look at. One panel per model, clearly labelled, with the date and project name, is what makes it usable a month later when nobody remembers the run. **PROMPT** — paste this into your AI coding assistant:

Add `build_report()` to `evaluate.py` that assembles one combined figure saved to `reports/evaluation_report.png` with a row per model - regression, classification, forecasting - showing the single most informative panel from each, with a clear row label and a shared title naming the course project and the date. Then write `reports/findings.md` with one short section per model containing: the headline metric next to its baseline, the most important thing the plots reveal that the metric does not, and one stated limitation of the model.

8. Read your own `findings.md` aloud as if presenting to a plant manager. If any sentence would not survive a follow-up question, rewrite it. The 'stated limitation' line

is not modesty, it is scope control. 'Trained only on machine types A and B; predictions for type C are not supported' is the sentence that prevents your model being misused, and it goes straight into the model card.

Test it

reports/eval_wear.png, reports/eval_defects.png and reports/eval_forecast.png each show three labelled panels. reports/evaluation_report.png combines one panel per model with a title and date. reports/findings.md has three sections, each naming a metric with its baseline, an insight the plot revealed that the metric did not, and a stated limitation.

Note: Full commands, prompts and sample code are in labs/lab20-*/README.md. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Lab 21 — Packaging a Complete Deep Learning Project

Learning outcome: package models, code, dependencies and documentation into a project another person can run.

Goal: Turn twenty labs of scripts into something you could hand to a colleague and walk away from. You restructure the workspace into a clean layout, pin the dependencies, write a predict.py command-line tool that loads any of the three saved models and scores new input, add a smoke test that proves the prediction path works end to end, and write the README and model cards. The real test is the last step: a partner clones your project into a fresh folder and runs it from the README alone, without asking you a single question.

What you'll build

A packaged, documented, tested ForgeSight project with a working CLI that another person can run unaided (Tools: Python packaging, pytest, git, Cursor / GitHub Copilot / Claude.)

Step-by-step

1. Restructure the workspace. Ask for a plan first rather than letting the assistant move files immediately. Asking for the plan before the action is the pattern to carry away from this lab. A restructure that moves thirty files and rewrites the imports is exactly the kind of change you want to review as a diagram first — reverting it afterwards is far more work than reading a tree. PROMPT — paste this into your AI coding assistant:

Review my torch-vibe project folder and propose a clean package structure for handover. Do NOT move anything yet - give me the proposed layout as a tree with a one-line purpose for each folder and each top-level file.

The project contains: data loading for tabular and image and series data; three trained models (tool-wear regression, defect CNN, vibration LSTM); a shared training engine; a checkpoint helper; evaluation scripts; saved checkpoints; and generated reports.

Constraints: source code separated from data, models and reports; every model loadable without running any training script; no absolute paths anywhere; data and model binaries excluded from version control but their absence explained in the README.

2. Review the proposed tree and adjust it before anything moves. You are the architect here, not the assistant. A reasonable layout: `src/` for data, models, engine and checkpoint code; `scripts/` for the training entry points; `tests/` for the smoke tests; `data/`, `models/` and `reports/` for artifacts; and `README.md`, `requirements.txt` and `.gitignore` at the root. Push back if the proposal buries the entry points.
3. Apply the restructure and fix the imports it breaks. Absolute paths are the most common reason a project fails on someone else's machine, and they are invisible on yours. `pathlib.Path(__file__).resolve().parent.parent` gives you a project root you can build every other path from. PROMPT — paste this into your AI coding assistant:

Apply the agreed structure. Move the files, update every import to match, and confirm each of the three training scripts and the evaluation script still run from the project root. Replace any absolute path with a path relative to the project root resolved via `pathlib`. Report every file you moved and every import you changed.

4. Write the prediction CLI — the thing that makes the project usable by someone who will never read your training code. This is the deliverable. Everything before this lab was for you; `predict.py` is for someone else. The clause about reading preprocessing statistics from the checkpoint is the direct payoff of Lab 11 — without it, this tool would feed raw values to a model trained on standardised ones and print confident nonsense, exactly as you demonstrated then. PROMPT — paste this into your AI coding assistant:

Create `predict.py`, a command-line tool for the packaged project.

```
USAGE: `python predict.py --model wear --input data/sample_reading.csv`, `python
predict.py --model defect --input path/to/image.png`, `python predict.py --model
vibration --input data/recent_series.csv`.
```

OPERATIONS: for each model - load the checkpoint, apply the SAME preprocessing the model was trained with by reading the saved statistics from the checkpoint, run inference under `torch.no_grad()` and `model.eval()`, and print a human-readable result. For the defect model print the predicted class name and its probability; for wear print the predicted microns; for vibration print the next predicted value and the persistence value alongside it for comparison.

CONSTRAINTS: validate the input file exists and has the expected columns or format, and fail with a clear message naming what is wrong rather than a traceback. Never re-fit any scaler - always use the statistics stored in the checkpoint. Include `--help` text for every argument.

EXPECTED OUTPUT: three working commands, each printing a labelled prediction.

5. Test all three commands yourself with the sample inputs. Test the error paths too, not just the happy ones. Point it at a file that does not exist and at a CSV with a missing column, and check the message names the actual problem. A tool that tracebacks at a colleague has not been handed over, it has been abandoned.

COMMAND — run this in your terminal:

```
python predict.py --model defect --input data/defects/val/scratch/scratch_001.png
```

```
python predict.py --model wear --input data/sample_reading.csv
```

```
python predict.py --model vibration --input data/vibration_series.csv
```

6. Add the smoke test that proves the prediction path works, so a future change that breaks it fails loudly. Test 5 is the one people leave out and the one that catches the most embarrassing failure: a model that loads, runs, returns the right shape and predicts the identical value for every input. All the other tests pass. Only varying the input reveals it. **PROMPT** — paste this into your AI coding assistant:

Create tests/test_predict.py using pytest.

TESTS: (1) each of the three checkpoints loads and returns a model in eval mode; (2) each model produces an output of the expected shape for a single synthetic input; (3) the defect model's softmax probabilities sum to 1 and the predicted class is a valid index; (4) predict.py exits with a clear non-zero status and a readable message for a missing input file; (5) the wear model's prediction changes when the input changes, which catches a model that always returns the same value.

CONSTRAINTS: tests must run without retraining anything and must not require internet access. Skip cleanly with a clear message if a checkpoint file is absent.

7. Run the tests and confirm they pass. Tests that need retraining or internet are tests nobody runs. Under a second, offline, is the target. **COMMAND** — run this in your terminal:

```
python -m pytest tests/ -v
```

8. Write the README and finalise the model cards, then pin the dependencies. The README's real audience is a capable stranger, which in practice is you in three months. If a step assumes something you happen to know today, it is a bug in the README. The results table with baselines is what stops anyone quoting your accuracy without its context. **PROMPT** — paste this into your AI coding assistant:

Write README.md for the packaged project covering: what ForgeSight does and the three models it contains; the exact setup commands from a fresh clone including creating the virtual environment and installing requirements; how to obtain or regenerate the data, since data files are not in version control; how to run each of the three predictions with a copyable example command; how to retrain each model; the project structure as a tree; and a results table of each model's headline metric next to its baseline. Also create .gitignore excluding .venv,

`__pycache__`, `data/`, `models/*.pt` and `reports/`. Then regenerate `requirements.txt` with pinned versions.

9. Run the handover test, which is the only verification that counts. The handover test: swap projects with a partner, clone into a fresh folder, and follow their README without speaking. Every question you have to ask is a defect in their documentation, and every question they ask is a defect in yours. This is the most useful fifteen minutes of the two days.

Test it

A partner clones your project into a fresh folder, follows README.md alone, creates the environment, installs from requirements.txt and successfully runs all three ``python predict.py`` commands without asking you anything. ``python -m pytest tests/ -v`` passes. No absolute path appears anywhere in the source, and each model has a model card stating its metric, its baseline and where it must not be used.

Note: Full commands, prompts and sample code are in `labs/lab21-*/README.md`. Read the AI-generated code before you run it, and use only data and accounts you are authorised to use. The ForgeSight telemetry, images and vibration series are synthetic and safe to share with an assistant.

Wrap-Up — What You Can Now Do

Across twenty-one labs you built a complete deep learning project with an AI pair programmer, from an empty folder to a packaged project covering tabular, image and sequence data. The `torch-vibe/` folder on your laptop is yours to keep, reuse and adapt at work.

The PyTorch skills

- Create, reshape, broadcast and move tensors, and read a shape error well enough to know which axis is wrong.
- Explain what autograd records, why `loss.backward()` works, and what `detach()` and `no_grad()` actually change.
- Build a network with `nn.Module`, choosing hidden activations by default and the output layer by the task.
- Match the loss to the output: `MSELoss` for regression, `CrossEntropyLoss` over raw logits for classification.
- Write the five-line training loop from memory, with `zero_grad` in the right place and `model.eval()` at evaluation.
- Build a CNN, reason about convolution, padding, stride and pooling, and fix a flatten-to-linear shape mismatch.
- Diagnose overfitting from the train/validation curves, then fix it with augmentation, dropout, weight decay and early stopping.

- Fine-tune a pre-trained ResNet by freezing the backbone and replacing the classifier head.
- Window a time series correctly, split it by time, and forecast with an LSTM that beats a persistence baseline.

The vibe coding skills

- Write a prompt that names the tensor shapes, the task, the layers, the constraints and the expected output.
- Read AI-generated PyTorch well enough to predict every printed shape before you run it.
- Recognise the classic AI PyTorch bugs: a softmax before CrossEntropyLoss, a missing zero_grad, augmentation applied to validation, a scaler fitted on the whole series, the wrong axis out of nn.LSTM.
- Feed a specific symptom back to the assistant instead of asking it to 'fix it'.
- Review and refactor a working-but-messy AI draft into modules you would be willing to sign your name to.
- Know when to stop prompting and read the PyTorch documentation instead.

Where this goes next

- Point the same pipeline at real data at work — machine telemetry, inspection images, or any sensor series.
- Swap the CNN for a modern backbone, or the LSTM for a Transformer, once the workflow is second nature.
- Move training onto a GPU or Colab when your dataset outgrows the CPU, changing only the device line.
- Add monitoring: track the live prediction distribution and retrain when it drifts away from the training data.

Next Steps

- First pass: complete every lab yourself, following the steps in this guide.
- Second pass: delete your scripts and rebuild Labs 8, 13 and 18 from memory, prompting from scratch.
- Point the models at a real dataset in your own organisation and see which of the twenty-one steps breaks first.
- Keep a prompts.md file of the prompts that worked — it becomes your personal deep learning toolkit.
- Explore the follow-on courses in machine learning, computer vision and NLP at www.tertiarycourses.com.sg.

Glossary

- **Activation function** — The non-linearity between layers - ReLU, sigmoid, tanh. Without it a stack of linear layers collapses into a single linear layer.
- **Autograd** — PyTorch's automatic differentiation engine. It records operations on requires_grad tensors into a graph and replays it backwards to compute gradients.
- **Backpropagation** — Applying the chain rule backwards through the computation graph to get the gradient of the loss with respect to every parameter.
- **batch_first** — The nn.LSTM/nn.GRU flag that selects (batch, seq, feature) instead of (seq, batch, feature). Getting it wrong silently trains on the wrong axis.
- **Broadcasting** — NumPy-style automatic expansion of tensor shapes in an elementwise operation. Convenient, and the reason a shape bug can run without error.
- **Computation graph** — The dynamic record of operations PyTorch builds on the forward pass, and consumes on the backward pass. Define-by-run means it is rebuilt every iteration.
- **Convolution** — Sliding a small learned kernel across an input so the same feature detector applies everywhere. The core operation of a CNN.
- **Cross entropy** — The loss for multi-class classification. nn.CrossEntropyLoss applies log-softmax internally, so the model must output raw logits.
- **Data augmentation** — Manufacturing new training views by randomly flipping, rotating, cropping or jittering inputs. Applied to the training set only.
- **DataLoader** — The PyTorch iterator that batches, shuffles and optionally parallel-loads a Dataset.
- **Dropout** — Randomly zeroing a fraction of activations during training to stop co-adaptation. Must be disabled at evaluation with model.eval().
- **Early stopping** — Halting training when validation loss stops improving, and keeping the best weights rather than the last ones.
- **Epoch** — One complete pass of the training data through the network.
- **Fine-tuning** — Continuing training a pre-trained network on your own data, often after freezing most of its layers.
- **GRU** — Gated Recurrent Unit - a recurrent cell with reset and update gates. Cheaper and faster than an LSTM, often just as accurate.
- **Learning rate** — The step size the optimizer takes along the gradient. The single hyperparameter most likely to be the reason training fails.
- **Logits** — The raw, unnormalised scores a classifier outputs before softmax. What nn.CrossEntropyLoss expects to be given.
- **Loss function** — The scalar the model minimises. It must match the output layer: MSELoss for regression, CrossEntropyLoss for multi-class classification.
- **LSTM** — Long Short-Term Memory - a recurrent cell with a cell state and forget, input and output gates, able to retain information over long sequences.
- **Lookback window** — The number of past timesteps a sequence model sees when predicting the next value or values.

- **model.eval() / model.train()** — The mode switch that turns dropout and batch-norm updating off and on. Forgetting it makes evaluation results wrong, not crash.
- **nn.Module** — The base class for every PyTorch model. Define layers in `__init__` and the computation in `forward()`.
- **no_grad()** — A context manager that stops autograd recording. Used at inference and evaluation to save memory and time.
- **Optimizer** — The rule that turns gradients into parameter updates - SGD follows the gradient, Adam adapts a per-parameter learning rate.
- **Overfitting** — Learning the training examples rather than the pattern. Recognised by a widening gap between training and validation loss.
- **Padding** — Adding a border of zeros before a convolution so the output keeps the input's spatial size.
- **Persistence baseline** — The naive forecast 'the next value equals the last value'. A sequence model that cannot beat it has not learned the series.
- **Pooling** — Downsampling a feature map by taking the maximum or mean over small windows, shrinking spatial size and adding a little translation tolerance.
- **PyTorch** — The define-by-run deep learning framework used throughout this course: tensors, autograd, nn.Module and an explicit training loop.
- **Regularization** — Anything that reduces overfitting - dropout, weight decay, augmentation, early stopping.
- **RNN** — Recurrent Neural Network - a network that carries a hidden state across timesteps, so order matters. Plain RNNs forget quickly.
- **Softmax** — The function that turns logits into probabilities summing to 1. Applied for reporting, not before `nn.CrossEntropyLoss`.
- **state_dict** — The dictionary of a model's learned tensors. The recommended thing to save - weights plus the class that built them, never a pickled object.
- **Stride** — How far the convolution kernel moves between positions. A stride above 1 downsamples.
- **Tensor** — An n-dimensional array with a dtype, a device and optionally a gradient history. The PyTorch data structure.
- **Transfer learning** — Reusing a network trained on a large dataset as the feature extractor for your own smaller problem.
- **Vibe coding** — Describing the outcome you want in plain language and letting an AI assistant write the code, while you keep control of the logic and verify the result.
- **Weight decay** — L2 regularization applied through the optimizer, penalising large weights and reducing overfitting.
- **zero_grad()** — Clearing accumulated gradients before `backward()`. Forgetting it silently sums gradients across batches and corrupts training.